

4. OpenGL

1. Introducere în OpenGL

Obiective ale capitolului

În acest capitol ne propunem:

- Să apreciem în mare ce se poate realiza cu OpenGL
- Să identificăm diferite nivele ale complexității reprezentării (rendering)
- Să înțelegem structura de bază a unui program OpenGL
- Să recunoaștem sintaxa comenzilor OpenGL
- Să identificăm secvența de operații a unui proces de renderizare (*rendering pipeline*)
- Să înțelegem în mare animațiile grafice folosind un program OpenGL

Acest capitol realizează o scurtă introducere în OpenGL, fiind împărțit în secțiunile următoare:

- **„Ce este OpenGL?”** explică ce este OpenGL, ce face și ce nu face, și cum funcționează.
- **„O scurtă prezentare a codului OpenGL”** prezintă un mic program OpenGL și analiza sa. Acest subcapitol definește de asemenea și câteva concepte legate de grafica asistată de calculator.
- **„Sintaxa comenzilor OpenGL”** explică unele dintre convențiile și notațiile folosite de comenzile OpenGL.
- **„OpenGL ca și mașină de stare”** descrie folosirea variabilelor de stare în OpenGL și comenzile de interogarea, activarea și dezactivarea stărilor.
- **„Procesul de renderizare OpenGL”** indică o secvență tipică a operațiilor pentru procesarea geometrică și a datelor unor imagini.
- **„Librării OpenGL”** descrie un set de rutine OpenGL, cât și o librărie dezvoltată special pentru a ușura folosirea OpenGL-ului.
- **„Animație”** explică în termeni generali cum se creează imagini și cum sunt acestea animate.

1.1 Ce este OpenGL?

OpenGL este o interfață software pentru accesul la resursele grafice ale unui calculator (ex: plăci grafice). Această interfață este alcătuită din circa 150 de comenzi distincte care sunt folosite pentru a specifica obiectele și operațiile folosite pentru a crea aplicații tridimensionale interactive. Este o interfață independentă atât de platforma hardware (PC, Mac) cât și de platforma software (sisteme de operare: Linux, Windows, OS2, Unix, etc...). Pentru a realiza acest lucru, OpenGL nu conține nici o comandă pentru a efectua operații pe ferestre sau pentru a obține date de la utilizator (tastatură, mouse). De asemenea, OpenGL nu pune la dispoziția programatorului comenzi de nivel înalt ce ar permite crearea unor forme relativ complicate, cum ar fi automobile, avioane sau molecule. Cu OpenGL, programatorul trebuie să-și construiască modelul dorit folosind un set mic de primitive geometrice: puncte, linii și poligoane.

O librărie creată special pentru a realiza aceste lucruri este „*OpenGL Utility Library*” sau pe scurt GLU. Este construită pe folosind OpenGL și pune la dispoziția programatorului o serie de facilități, printre care amintim de suprafețele cuadrice, curbele și suprafețele NURBS. GLU este o parte standard din orice implementare OpenGL.

Ce se poate realiza cu OpenGL?

1. Construirea unor obiecte folosind primitive grafice, în consecință folosind descrieri matematice a obiectelor (punctele, liniile, poligoanele, imaginile și bitmap-urile sunt considerate în OpenGL primitive).
2. Aranjează obiectele într-un spațiu tridimensional și selectează punctul de vizualizare a scenei create.
3. Calculează culorile tuturor obiectelor. Culoarea poate fi atribuită explicit de către aplicație, determinată din condiții specifice de iluminare, obținută prin atribuirea unor texturi obiectelor sau prin combinarea acestor operațiuni.
4. Converstește descrierile matematice ale obiectelor și culorile asociate lor în pixeli pe ecran. Acest proces este cunoscut sub numele de rasterizare.

În timpul acestor etape, OpenGL ar mai putea efectua și alte operații cum ar fi eliminarea părților unui obiect ce sunt ascunse de către alte obiecte. În plus, după ce scena este rasterizată dar înainte de a fi desenată pe ecran, dacă mai dorește, programatorul ar mai putea efectua operații pe pixelii obținuți în urma rasterizării.

1.2. O scurtă prezentare a codului OpenGL

Deoarece se pot realiza atât de multe lucruri cu un sistem grafic OpenGL, un program OpenGL poate fi destul de complicat. Totuși structura de bază a unui program folositor poate fi relativ simplă: scopul său este de a inițializa anumite stări ce controlează modul cum OpenGL renderizează o scenă și de a specifica obiectele ce se doresc a fi renderizate.

Înainte de a prezenta cod sursă scris folosind OpenGL, se impune a defini unii termeni de bază folosiți cu precădere în aplicațiile grafice.

Renderizare (rasterizare) – este procesul prin care un calculator creează imagini pe baza unor modele matematice. Aceste modele, sau obiecte, sunt construite din primitive geometrice (puncte, linii și poligoane) și sunt descrise prin vertex-urile aferente.

Vertex – un punct din spațiu prin care trec părțile unui obiect (de obicei este încetățenit sub numele de vârf), descris matematic printr-un set de două coordonate (x, y) pentru spațiul bidimensional sau un set de trei coordonate (x, y, z) pentru spațiul tridimensional.

Imaginile renderizate în final sunt constituite din pixeli desenați pe ecran.

Pixel – este cel mai mic element vizibil care poate fi afișat pe ecran de către o placă grafică. Informațiile despre pixeli (ex: culoarea lor) sunt organizate în memorie sub forma unor plane de biți.

Plane de biți (bitplanes) – sunt locații de memorie ce conțin un bit de informație pentru fiecare pixel de pe ecran; de exemplu bit-ul ar putea indica cât de galben ar trebui să fie un pixel de pe ecran. Planele de biți sunt organizate în buffere de cadre (*framebuffer*), ce conțin toată informația necesară dispozitivelor grafice pentru a controla culoarea și intensitatea tuturor pixelilor de pe ecran.

În continuare în Exemplul 1.1 se prezintă un program simplu ce folosește OpenGL pentru a desena un dreptunghi alb pe un fundal negru (Figura 1.1).

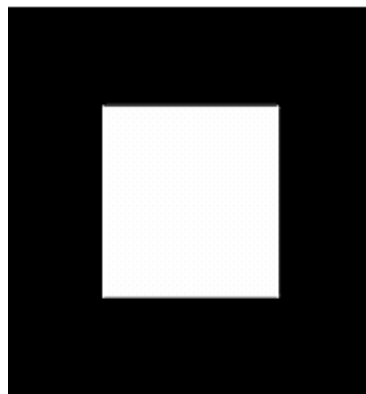


Figura 1.1 Dreptunghi alb pe fundal negru

Exemplul 1.1

```
#include "biblioteci necesare"
void main() {
    initializeazaFereastra();
    glClearColor (0.0, 0.0, 0.0, 0.0);
    glClear (GL_COLOR_BUFFER_BIT);
    glColor3f (1.0, 1.0, 1.0);
    glOrtho(0.0, 1.0, 0.0, 1.0, -1.0, 1.0);
    glBegin(GL_POLYGON);
    glVertex3f (0.25, 0.25, 0.0);
    glVertex3f (0.75, 0.25, 0.0);
    glVertex3f (0.75, 0.75, 0.0);
    glVertex3f (0.25, 0.75, 0.0);
    glEnd();
    glFlush();
    actualizeazaFereastraSiVerificaEventualeleEvenimente();
}
```

Prima linie de cod din rutina **main()** inițializează o fereastră pe ecran; rutina **initializeazaFereastra()** se dorește a fi locul în care se accesează funcțiile specifice sistemului de operare pe care se execută programul și care în general nu sunt definite în OpenGL. Următoarele două linii de cod sunt comenzi OpenGL care șterg ecranul cu culoare de fond neagră; **glClearColor()** stabilește ce culoare va avea ecranul după ștergere, iar **glClear()** șterge ecranul efectiv. Odată ce culoarea de fond este setată, de fiecare data când este apelată funcția **glClear()**, fereastra este afisată folosind această culoare. Această culoare de ștergere poate fi schimbată doar printr-un nou apel al funcției **glClearColor()**. În mod similar funcția **glColor3f()** stabilește cu ce culoare se vor desena obiectele pe ecran, în cazul nostru cu culoare albă. Toate obiectele desenate după acest punct vor folosi această culoare, până când este schimbată printr-o apelare nouă a funcției **glColor3f()**.

Următoarea comandă OpenGL folosită în program este **glOrtho()**, comandă ce specifică sistemul de coordonate folosit de OpenGL atunci când va desena imaginea finală pe ecran. Următoarele apeluri, ce sunt delimitate de către **glBegin()** și **glEnd()**, definesc obiectul ce va fi desenat, în acest exemplu un poligon cu patru vertex-uri. Colțurile poligonului sunt definite de către comanda **glVertex3f()**. După cum se poate observa din coordonatele furnizate, poligonul este un dreptunghi în planul $z = 0$. În final, **glFlush()** asigură faptul că comenzile sunt în fapt executate imediat și nu sunt stocate într-un buffer ce așteaptă comenzi OpenGL adiționale. Funcția **actualizeazaFereastraSiVerificaEvenimentele()** se ocupă de conținutul unei ferestre și de procesarea evenimentelor generate de către ea.

Mai târziu, în acest capitol, se vor prezenta două funcții, echivalente funcțiilor **initializeazaFereastra()** și **actualizeazaFereastraSiVerificaEvenimentele()**, specifice sistemului de operare Windows, dar va fi nevoie de o restructurare a codului prezentat mai sus doar cu scop de introducere.

1.3. Sintaxa comenzilor OpenGL

După cum probabil s-a observat în programul exemplificat mai sus, comenzile OpenGL folosesc ca și prefix **gl** și literă mare pentru fiecare cuvânt ce alcătuiește numele unei comenzi (ex: **glClearColor()**).

În mod similar OpenGL definește constantele ca începând cu **GL_**, toate literele mari și folosește „_” pentru a separa cuvintele (ex: **GL_COLOR_BUFFER_BIT**).

Adițional, unele comenzi mai prezintă litere în plus la sfârșitul numelui, indicând tipul de date al parametrilor care îi primește funcția (ex. **3f** din comenzile **glColor3f()** și **glVertex3f()**). Unele comenzi OpenGL accepta până la 8 tipuri diferite de date pentru argumentele lor. Literele folosite la sufixarea acestor comenzi sunt definite conform standardului ISO C și sunt prezentate în Tabelul 1.1, împreună cu tipul de date corespunzător în OpenGL.

Tabel 1.1: Sufixurile unei comenzi și tipurile de date corespundente

Sufix	Tipul de dată	Limbaj C	OpenGL
b	întreg pe 8 biți	signed char	GLbyte
s	întreg pe 16 biți	short	GLshort
i	întreg pe 32 de biți	int sau long	GLint, GLsizei
f	flotant pe 32 de biți	float	GLfloat, GLclampf
d	flotant pe 64 de biți	double	GLdouble, GLclampd
ub	întreg pozitiv pe 8 biți	unsigned char	GLubyte, GLboolean
us	întreg pozitiv pe 16 biți	unsigned short	GLushort
ui	întreg pozitiv pe 32 biți	unsigned int sau unsigned long	GLuint, GLenum, GLbitfield

Astfel, comenzile

```
glVertex2i(1, 3);
```

```
glVertex2f(1.0, 3.0);
```

sunt echivalente, exceptând faptul că prima specifică coordonatele vertexului ca întregi pe 32 de biți, iar a doua ca numere flotante în precizie simplă.

Unele comenzi OpenGL pot avea ca litera finală litera „v”, lucru ce indică faptul că funcția ia ca și parametru un pointer la un vector de valori și nu o serie de argumente valorice individuale.

Ex.

```
glColor3f(1.0, 0.0, 0.0);
```

```
GLfloat color_array[] = {1.0, 0.0, 0.0};
```

```
glColor3fv(color_array);
```

În final OpenGL definește tipul de dată vid ca fiind GLvoid.

În restul capitolului, comenzile OpenGL sunt referite după numele lor de bază și mai includ un asterix pentru a indica faptul că acolo ar putea fi mai multe indicații legate de tipurile de date ale argumentelor ce le poate primi. De exemplu, **glColor*()** reprezintă toate variațiile posibile ale comenzii folosite pentru a seta culoarea (ex. **glColorf()**, **glColori()**, etc...). Notăția **glVertex*v()** se referă la toate versiunile comenzii ce acceptă vectori cu tipuri de date diferite (ex. **glVertexfv()**, **glVertexiv()**, etc...).

1.4. OpenGL ca și mașină de stare

OpenGL este o mașină de stare. Se pot seta stări sau moduri variate și vor rămâne active până vor fi schimbate de către aplicație. De exemplu, se poate observa că funcția ce setează culoarea curentă, inițializează o variabilă de stare ce rămâne astfel pe tot parcursul execuției programului până este schimbată din nou. Alte variabile de stare ar fi transformările curente de vizualizare și proiecție, modurile de desenare a unui poligon, convențiile la nivelul de împachetare a pixelilor, pozițiile și caracteristicile luminilor, materialelor. Multe dintre variabilele de stare pot fi activate, respectiv dezactivate folosind comenzile **glEnable()** și **glDisable()**.

Fiecare variabilă de stare sau mod au o valoare predefinită și în orice moment programatorul poate afla valoarea curentă a lor. De obicei, se poate folosi una dintre următoarele șase comenzi: **glGetBooleanv()**, **glGetDoublev()**, **glGetFloatv()**, **glGetIntegerv()**, **glGetPointerv()** sau **glIsEnabled()**. Folosirea uneia dintre comenzile anterioare depinde de tipul de dată dorit pentru răspunsul căutat. Unele variabile de stare au una sau mai multe comenzi specifice, cum ar fi: **glGetLight*()**, **glGetError()** sau

glGetPolygonStipple(). În plus, se poate salva o colecție de variabile de stare într-o stivă de attribute folosind comenzile **glPushAttrib()** sau **glPushClientAttrib()**, să fie modificate temporar, și mai târziu readuse la vechea valoare folosind **glPopAttrib()** sau **glPopClientAttrib()**.

1.5 Procesul de renderizare OpenGL

Majoritatea implementărilor OpenGL au o ordine a operațiilor similară, o serie de etape de procesare ce poartă numele de țeava de renderizare (**rendering pipeline**). Această ordine, după cum este ilustrată în Figura 1.2, nu este o regulă strictă despre cum OpenGL poate fi implementat, dar oferă un ghid sigur despre ce face OpenGL într-un anumit moment al execuției aplicației.

Următoarea diagramă indică modul în care OpenGL trece datele ce trebuiesc procesate printr-o serie de operații necesare pentru a realiza reprezentarea grafică dorită. Datele geometrice (vertex-uri, linii și poligoane) urmează calea prin diferite „cutii” ce includ printre altele, evaluatori și operații la nivel de pixel, în timp ce datele despre pixeli (pixeli, imagini și bitmap-uri) sunt tratate diferit în anumite părți ale procesului de renderizare. Ambele tipuri de date trec spre final prin aceeași pași (rasterizare și operații pe fragmente) înainte de a fi scrise în buffer-ul de cadre.

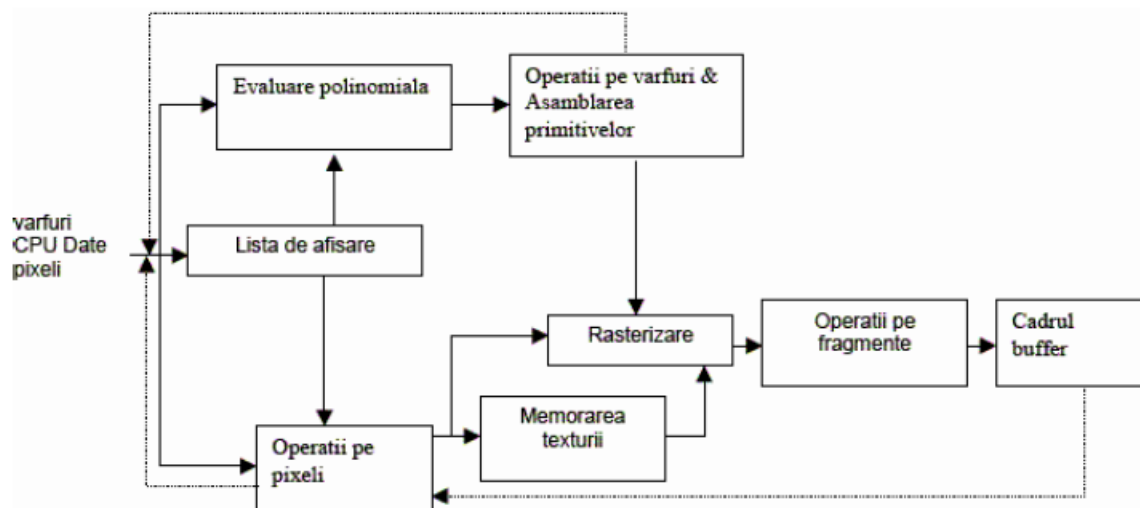


Figura 1.2 Ordinea operațiilor OpenGL

1.5.1 Lista de afisare (display)

Toate datele, fie că descriu geometria sau pixelii, pot fi salvate într-o *lista display*, atât pentru utilizarea curentă a datelor cât și pentru o utilizare ulterioară. (Alternativa de a păstra datele într-o *lista display* este de a procesa datele imediat - mod cunoscut ca „*immediate*”

mode”). Când o *listă display* este executată, datele păstrate în acesta sunt transmise ca și cum ar fi fost transmise de aplicație în mod imediat.

1.5.2 Evaluare polinomială (Evaluatori)

Toate primitivele geometrice sunt descrise în cele din urmă prin vârfuri. Curbele parametrice și suprafețele pot fi inițial descrise prin puncte de control și funcții polinomiale numite funcții de bază. Evaluările furnizează o metodă de derivare a vârfurilor utilizate pentru a reprezenta suprafața din puncte de control. Metoda este o map-are polinomială, ce poate reproduce suprafața normal, coordonatele texturii, culorile și coordonatele spațiale din punctele de control.

1.5.3 Operațiile pe vertex-uri

Operațiile pe vertex-uri au ca și scop convertirea datelor geometrice despre vertex-uri în primitive. Unele date despre vertex-uri (de exemplu: coordonatele spațiale) sunt transformate în matrici de 4x4. Coordonatele spațiale sunt proiectate dintr-o poziție din lumea tridimensională într-o poziție de pe ecranul nostru.

1.5.4 Construirea primitivelor

Tăierea (decuparea), o parte majoră în construirea primitivelor, reprezintă eliminarea porțiunilor geometrice care apar în exteriorul spațiului definit de un plan. Tăierea punctelor presupune eliminarea de vertex-uri; tăierea liniilor și a poligoanelor presupune adăugarea de vertex-uri adiționale, depinzând de modul în care este tăiată linia sau poligonul.

În unele cazuri, această etapă este urmată de divizarea perspectivei, care produce efectul ca obiectele aflate la distanță mai mare să apară mai mici decât obiectele aflate la o distanță mai mică de privitor. Apoi, sunt aplicate operații ale punctului de vedere (viewport) și la ce adâncime (coordonata z). Depinde de tipul poligonului, acesta poate fi desenat ca puncte sau linii.

Rezultatele acestei etape sunt primitivele geometrice complete, care sunt de fapt vertex-urile transformate și tăiate ce au o culoare și o adâncime proprie. Acestea mai sunt câteodată și valori ale coordonatelor texturilor și linii de ghidaj pentru procesul de rasterizare.

1.5.5 Operații pe pixeli

În timp ce datele geometrice iau o anumită cale în procesul de renderizare în OpenGL, datele despre pixeli iau o altă cale. Pixelii dintr-o matrice din memoria sistemului sunt mai întâi transformați din una din formatele existente într-un număr de componente potrivit. În următoarea etapă datele sunt scalate și procesate de către harta de pixeli. Rezultatele sunt fie scrise în memoria unde sunt păstrate datele despre texturi, fie sunt trimise către procesul de rasterizare.

Dacă datele despre pixeli sunt citite din buffer-ul de cadre, atunci au loc o serie de operații asupra pixelilor (scalare, mapare, etc.). Mai apoi aceste rezultate sunt împachetate într-un format potrivit și sunt returnate în matricea din memoria sistemului. Mai există și alte operații speciale de copiere a pixelilor, cum ar fi copierea datelor din buffer-ul de cadre către alte porțiuni din același buffer sau în memoria unde sunt păstrate texturile.

1.5.6 Construirea texturilor

Unele aplicații OpenGL ar putea dori să aplice texturi (imagini) pe anumite obiecte geometrice pentru a le face să arate mai realistic. Dacă sunt folosite mai multe texturi este recomandat să fie puse în obiecte de texturi astfel încât să fie ușor interschimbabile între ele.

Unele implementări ale OpenGL-ului pot folosi resurse speciale pentru a crește performanța operațiilor efectuate asupra texturilor. Aceste resurse ar putea fi memorii de performanță mare, super specializate în operații pe texturi. Dacă aceste tipuri de memorie sunt disponibile atunci obiectele de texturi pot fi setate să controleze folosirea acestora.

1.5.7 Rasterizarea

Rasterizarea este procesul de conversie a datelor geometrice și pixelilor în fragmente pătrate. Fiecărui fragment îi corespunde un pixel din buffer-ul de cadre. Modelele de linii și poligoane, lățimea liniei, dimensiunea punctului, modelul de umbrire și calculele de a suporta antialiasing (metodă pentru netezirea marginilor zimțate). Sunt luate în considerare ca și vertex-uri ce formează linii sau ca și pixelii folosiți pentru a umple interiorul unui poligon. Pentru fiecare fragment pătratic sunt atribuite valori de culoare și adâncime.

1.5.8 Operații pe fragmente

Înainte ca valorile să fie stocate în buffer-ul de cadre, sunt realizate o serie de operații care ar putea altera sau chiar elimina unele fragmente. Toate aceste operații pot fi activate sau dezactivate.

Prima operație care poate fi întâlnită este texturarea, unde un texel (element al texturii) este generat din memoria texturii pentru fiecare fragment și aplicat fiecărui fragment. Apoi s-ar mai putea aplica calcule pentru crearea efectului de ceață, urmată de o serie de teste printre care amintim de testul buffer-ului de adâncime (buffer-ul este folosit pentru înlăturarea suprafețelor ascunse). Dacă un anumit test eșuează atunci procesarea acestui fragment s-ar putea opri. Apoi operații de blending, dithering, operații logice, ascunderea folosind o mască de biți, pot fi unele dintre operațiile ce s-ar mai putea efectua. În final, fragmentul procesat este stocat într-un buffer potrivit, unde în sfârșit a avansat la stadiul de pixel.

1.6. Librării OpenGL

OpenGL oferă un set de comenzi puternice dar primitive și toate operațiile de desenare de nivel înalt trebuie făcute folosind aceste comenzi. De asemenea programele OpenGL sunt forțate să folosească mecanismele sistemului de ferestre a sistemului de operare pe care rulează acestea. Există în domeniu o serie de librării ce ușurează programarea OpenGL, printre care enumerăm următoarele:

- Librăria de utilități OpenGL (OpenGL Utility Library – **GLU**) conține diferite rutine ce folosesc comenzi OpenGL de nivel inferior pentru a efectua operații de setare a matricilor pentru orientarea anumitor puncte de vedere sau proiecții pentru crearea poligoanelor și randarea suprafețelor. Această librărie este integrată în OpenGL. Comenzile din GLU sunt descrise în manualul de referință al OpenGL-ului și sunt prefixate de literele **glu**.
- Pentru fiecare sistem de ferestre există o librărie care extinde funcționalitatea acestui sistem pentru a suporta renderizarea OpenGL. Pentru mașinile care folosesc X Window System, extensia OpenGL (**GLX**) pentru acest tip de sisteme este oferită ca un complement pentru OpenGL. Comenzile GLX sunt prefixate de literele **glX**. Pentru Microsoft Windows comenzile WGL oferă ferestre interfeței OpenGL. Toate comenzile WGL sunt prefixate **wgl**. Pentru IBM OS/2, este folosit PGL (Presentation Manager) și comenzile acestei extensii sunt prefixate de **pgl**.
- OpenGL Utility Toolkit (**GLUT**) este o librărie independentă de sistemul de ferestre, scrisă de către Mark Kilgard, pentru a ascunde din complexitatea API-urilor folosite pentru crearea ferestrelor pe sisteme diferite de operare.
- **Open Inventor** este un pachet de utilități orientate pe obiect și bazate pe OpenGL ce oferă o serie de obiecte și metode pentru a crea aplicații tridimensionale interactive. Open Inventor (scris în limbajul de programare C++) oferă obiecte predefinite și modele de evenimente pentru interacțiunea cu utilizatorul, componente de nivel înalt pentru crearea și editarea scenelor tridimensionale, și abilitatea de a afișa obiecte și interschimba datele în alte formate grafice. Open Inventor este oferit ca pachet separat de OpenGL.

1.6.1 Fișierele header ce trebuie incluse în aplicații OpenGL

Pentru toate aplicațiile OpenGL este necesar să se includă headerul **gl.h** în fiecare fișier unde se folosesc comenzi OpenGL. Aproape toate aplicațiile OpenGL folosesc GLU, librărie ce necesită folosirea fișierului header **gl.h**. Astfel fiecare fișier sursă OpenGL începe cu

```
#include "GL/gl.h"
```

```
#include "GL/glu.h"
```

Dacă se accesează o librărie pentru o interfață de ferestre în mod direct este necesară, pentru suportul OpenGL, includerea unor headere adiționale. De exemplu, pentru Microsoft Windows se folosește:

```
#include "windows.h"
```

```
#include "GL/gl.h"
```

```
#include "GL/glu.h"
```

Dacă în aplicație se va folosi GLUT atunci ar mai fi necesară includerea fișierului:

```
#include "GL/glut.h"
```

De observat că **glut.h** include automat și headerele **gl.h** și **glu.h** astfel încât includerea lor încă o dată ar fi redundantă. GLUT pentru Microsoft Windows include și windows.h.

1.6.2 GLUT (OpenGL Utility Toolkit)

După cum se știe, OpenGL conține comenzi pentru randare, dar este proiectat să fie independent de platforma pe care rulează, astfel încât nu conține nici o comandă pentru crearea ferestrelor sau pentru citirea evenimentelor generate de tastatură sau mouse. GLUT oferă posibilitatea acestor operații și în plus mai oferă posibilitatea creării obiectelor grafice complicate (sfere, torus și teapot), obiecte ce OpenGL nu le oferă nativ. Restul subcapitolului oferă o scurtă introducere în comenzile oferite de GLUT.

1.6.2.1 Managementul ferestrelor

Cinci rutine sunt necesare pentru inițializarea unei ferestre:

- **glutInit(int *argc, char **argv)** inițializează GLUT și procesează argumente oferite în linia de comandă la execuția programului. **glutInit()** trebuie să fie apelată înaintea apelului oricărei alte comenzi GLUT.
- **glutInitDisplayMode(unsigned int mode)** specifică dacă se va folosi un model de culoare bazat pe RGBA sau un index de culoare. De asemenea se poate specifica folosirea pentru afișarea a unui sistem de buffer unic sau buffer dublu.
- **glutInitWindowPosition(int x, int y)** specifică poziția colțului din stânga-sus a ferestrei.
- **glutInitWindowSize(int width, int height)** specifică dimensiunea ferestrei în pixeli.

- **int glutCreateWindow(char *string)** creează o fereastră în context OpenGL și returnează un identificator unic. Fereastra va fi afișată doar în momentul când este apelată funcția **glutMainLoop()**.

1.6.2.2 Callback-ul de afișare

glutDisplayFunc(void (*func)(void)) este prima și cea mai importantă funcție ce este apelată în urma apariției unui eveniment ce necesită schimbarea contextului afișat pe ecran. De fiecare dată când GLUT determină că conținutul unei ferestre trebuie reafiat, funcția înregistrată cu **glutDisplayFunc()** va fi executată. Astfel, toate rutinele ce se ocupă de desenarea conținutului unei ferestre ar trebui puse în interiorul funcției înregistrată cu **glutDisplayFunc()**. Dacă programul schimbă conținutul unei ferestre, uneori este necesar să apelăm funcția **glutPostRedisplay()** ce anunță funcția **glutMainLoop()** să apeleze funcția înregistrată pentru afișare cât mai devreme posibil.

1.6.2.3 Execuția unui program

În final, după ce au avut loc toate inițializările ferestrei și alte setări necesare se apelează funcția **glutMainLoop()**. Toate ferestrele care au fost create, vor fi acum afișate, și renderizare în aceste ferestre devine posibilă. Operațiile pentru randarea scenei se află în funcția **display()** ce este înregistrată de către GLUT ca fiind funcția callback de afișare. Pentru exemplificare folosim programul din Exemplul 1.2

Exemplul 1.2 : Un simplu program folosind GLUT: hello.c

```
#include <windows.h>
#include <GL/gl.h>
#include <GL/glut.h>
void display(void)
{
    /* șterge toți pixelii */
    glClear (GL_COLOR_BUFFER_BIT);
    /* desenează un dreptunghi cu colțurile în punctele
    * (0.25, 0.25, 0.0) și (0.75, 0.75, 0.0)
    */
    glColor3f (1.0, 1.0, 1.0);
    glBegin(GL_POLYGON);
    glVertex3f (0.25, 0.25, 0.0);
    glVertex3f (0.75, 0.25, 0.0);
    glVertex3f (0.75, 0.75, 0.0);
    glVertex3f (0.25, 0.75, 0.0);
}
```

```

    glEnd();
    /* forțează procesarea rutinelor de desenare */
    glFlush ();
}
void init (void)
{
    /* selectează culoarea de ștergere a ecranului */
    glClearColor (0.0, 0.0, 0.0, 0.0);
    /* inițializează punctul de vizualizare al scenei */
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(0.0, 1.0, 0.0, 1.0, -1.0, 1.0);
}
/*
* Declara dimensiunea inițială a ferestrei și modul de afișare
* (single buffer si RGBA). Deschide fereastra cu "hello" in bara
* de titlu. Apelează rutinele de inițializare. Înregistrează funcțiile
* callback de afișarea. Intra în bucla principală si procesează
* eventualele evenimente.
*/
int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize (250, 250);
    glutInitWindowPosition (100, 100);
    glutCreateWindow ("hello");
    init ();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

1.6.2.4 Tratarea evenimentelor de intrare (tastatură, mouse)

Pentru a înregistra funcții callback pentru a fi apelate în momentul când au loc anumite evenimente de intrare se pot folosi următoarele comenzi GLUT:

- **glutReshapeFunc(void (*func)(int w, int h))** indică ce acțiuni sunt necesare a se efectua în cazul în care fereastra este redimensionată.

- **glutKeyboardFunc**(void (*func)(unsigned char key, int x, int y)) și **glutMouseFunc**(void (*func)(int button, int state, int x, int y)) permit programatorului să lege o tastă sau un buton de la mouse cu o rutină ce este invocată de fiecare dată când tasta sau butonul mouse-ului sunt apăstate sau eliberate.
- **glutMotionFunc**(void (*func)(int x, int y)) înregistrează o funcție callback ce va fi apelată în momentul când mouse-ul este mișcat sau se apasă unul dintre butoanele mouse-ului.

1.6.2.5 Administrarea unui proces

Se mai poate specifica o funcție callback ce va fi apelată atunci doar atunci când nu există nici un eveniment care așteaptă să fie tratat, folosind comanda **glutIdleFunc**(void (*func)(void))).

1.6.2.6 Desenarea obiectelor tridimensionale

GLUT include diferite comenzi pentru desenarea obiectelor tridimensionale, printre care comenzi pentru generarea următoarelor obiecte:

- conuri
- cuburi
- dodecahedron
- icosahedron
- octahedron
- sfere
- teapot
- tetrahedron
- torus

Aceste obiecte se pot desena ca și plase (wireframe) sau ca obiecte solide, normalele la suprafață definite. De exemplu, pentru crearea unui cub și a unei sfere se pot folosi următoarele comenzi:

- **void glutWireCube(GLdouble size)**
- **void glutSolidCube(GLdouble size)**
- **void glutWireSphere(GLDouble radius, GLint slices, GLint stacks)**
- **void glutSolidSphere(GLDouble radius, GLint slices, GLint stacks)**

Toate aceste modele sunt desenate în centrul originii sistemului de coordonate a lumii (scenei).

1.7. Animație

Unul dintre lucrurile cele mai importante atunci când se discută despre grafica pe calculator îl reprezintă animația imaginilor.

Într-un cinema, animația este obținută prin proiectarea unei secvențe de imagini la o rată de 24 de frame-uri pe secundă. Chiar dacă privitorul privește 24 de imagini diferite într-o secundă, creierul omenesc le unește într-o animație continuă. În realitate, cele mai moderne proiectoare afișează fiecare imagine de două ori, astfel obținându-se o rată de 48 de frame-uri pe secundă, înlăturându-se efectul de clipire. Monitoarele calculatoarelor, de obicei reîmprospătează imaginile cu o rată de 60 până la 70 de imagini pe secundă, unele ajungând chiar până la 120 de reîmprospătări pe secundă (rată cunoscută sub termenul tehnic de **refresh**). În mod clar 60 de reîmprospătări pe secundă sunt mult mai bine decât 30 pe secundă, iar 120 cu mult mai bine. Totuși o rată a refresh-ului mai mare de 120 pe secundă, nu mai are sens deoarece ochiul omenesc nu mai percepe nici o diferență.

În concluzie, tehnica de creare a animațiilor funcționează deoarece fiecare frame este complet înainte de a fi afișat. Să presupunem ca se dorește a se crea o animație de un milion de frame-uri cu un program după cum este ilustrat mai jos:

```
open_window ();  
for (i = 0; i < 1000000; i++) {  
    clear_the_window ();  
    draw_frame (i);  
    wait_until_a_24th_of_a_second_is_over ();  
}
```

Dacă adăugați timpul necesar pentru ca sistemul să șteargă ecranul și să afișeze un frame, acest program va da rezultate din ce în ce mai puțin bune depinzând de cât de bine reușește într-o 1/24 secunde să șteargă și să afișeze noul frame. Dacă presupunem că afișarea unui frame durează aproape 1/24 dintr-o secundă, atunci fragmente de început al unui frame sunt afișate timp de 1/24 secunde într-o formă solidă, dar fragmentele de la sfârșitul frame-ului ce sunt afișate mai târziu sunt instantaneu șterse pentru că programul avansează la următorul frame. Astfel proiecția va avea ca rezultat apariția unor bucăți de imagine cu efect de clipire, deoarece în loc să privim o imagine completă, noi privim crearea unei imagini pe dispozitivul grafic. Astfel este necesară o modalitate prin care programul să afișeze frame-uri complete înainte de a le înlocui cu noi frame-uri.

Majoritatea implementărilor OpenGL au sisteme de dublu buffer (la nivel hardware sau software) ce oferă două buffere de culoare complete. Unul este afișat în timp ce în celălalt se

desenează. Când desenarea unui frame este completă atunci cei doi bufferi sunt interschimbați, astfel încât cel ce era vizualizat acum este folosit pentru desenat și vice versa. Cu buffere duble, fiecare frame este afișat doar atunci când procesul de desenare este complet, astfel încât privitorul nu va vedea niciodată frame-uri parțial desenate.

O variantă modificată a programului anterior, ce realizează o afișare cursivă și completă a animației s-ar putea scrie în felul următor:

```
open_window_in_double_buffer_mode();  
for (i = 0; i < 1000000; i++) {  
    clear_the_window();  
    draw_frame(i);  
    swap_the_buffers();  
}
```

Refresh-ul cu pauză

Pentru unele implementări OpenGL, pe lângă schimbarea celor doi bufferi, de desenare și respectiv de afișare, între ei, funcția **swap_the_buffers()** mai așteaptă până când perioada de re-împrospătare a ecranului este încheiată astfel încât buffer-ul anterior să fie afișat complet. Această rutină permite de asemenea ca noul buffer să fie complet afișat, pornind cu începutul. Presupunând ca un sistem reîmprospătează ecranul de 60 de ori pe secundă, acest lucru înseamnă rata maximă de afișare a frame-urilor pe ecran poate fi doar de 60 fps (frame-uri pe secundă), și dacă toate frame-urile pot fi șterse și afișate în mai puțin de 1/60 secunde atunci vom avea ca rezultat o animație fluidă și continuă.

De obicei se întâmplă ca pe astfel de sisteme, frame-urile să fie prea complicate pentru a putea fi afișate într-o perioadă de 1/60 secunde. De exemplu, dacă este necesar 1/45 secunde pentru a afișa un frame, rezultatul va fi de 30 fps, și sistemul grafic intră într-o stare de așteptare, $1/30 - 1/45 = 1/90$ secunde pentru fiecare frame sau o treime din timp.

În plus rata de re-împrospătare a filmului este constantă, ducând uneori la rezultate nedorite. De exemplu, cu un monitor cu refresh-ul de 60 (1/60 secunde pentru re-împrospătarea ecranului) și o rată de frame-uri constantă, se pot obține 60, 30, 20, 15, 12 fps (60/1, 60/2, 60/3, 60/4, ...). Acest lucru înseamnă că dacă se dezvoltă o aplicație la care se adaugă constant tot mai multe îmbunătățiri (ex. **un simulator de zbor la care se adaugă terenuri**), la început fiecare îmbunătățire adusă sistemului s-ar putea să nu influențeze performanța aplicației, dar la un moment dat s-ar putea ca ea să scadă brusc de la 60 fps la 30 fps deoarece sistemul nu mai este capabil să deseneze și să afișeze un frame în 1/60 secunde. La fel s-ar putea întâmpla pentru o rată de 30 fps să scadă la 20 fps.

Dacă complexitatea unei scene determină ca desenarea și afișarea unui frame să dureze o perioadă de timp apropiată de perioadele “magice” (1/60 secunde, 2/60 secunde, 3/60 secunde, ...), atunci datorită unor variații aleatoare s-ar putea ca rata de afișare să varieze ușor în jurul acestor perioade “magice”. Astfel rata de afișare a frame-urilor va varia în timp, putând deveni din punct de vedere vizual deranjant. În acest caz, dacă scena nu s-ar putea simplifica astfel încât toate frame-urile să fie la fel de rapide, este mai bine să se impună o anumită întârziere în afișarea frame-urilor, pentru a asigura o rată a fps-urilor constantă.

Animație = Redesenare + Interschimbarea Buffer-elor

Structura adevăratelor programe de animație nu diferă prea mult de această descriere. De obicei, este mai ușor să se redeseneze buffer-ul în întregime pentru fiecare frame decât să se detecteze care părți din scenă sau modificat și numai ele să fie redesenat.

În cele mai multe animații, obiecte dintr-o scenă sunt de obicei doar redesenat cu diferite transformări (punctul de vedere al privitorului se modifică), sau o mașină își modifică poziția, sau un obiect este rotit cu un unghi mic. Dacă este necesar un timp mare pentru realiza diferite calcule non-grafice, atunci se observă o scădere a fps-ului. A se reține totuși că perioada de așteptare ce apare după ce se execută **swap_the_buffers()** poate fi folosită pentru astfel de calcule.

OpenGL nu are o comandă **swap_the_buffers()** deoarece această facilități ar putea să nu fie disponibilă pe toate dispozitivele grafice (plăci grafice) și oricum o astfel de funcție ar fi foarte dependentă de sistemul de operare (OpenGL se vrea complet independent de platforma pe care rulează).

Dacă se folosește pachetul de utilități GLUT, acesta pune la dispoziția utilizatorilor o funcție care realizează acest lucru: **void glutSwapBuffers(void)**.

Exemplul 1.3 ilustrează folosirea comenzii **glutSwapBuffer()** într-un exemplu ce desenează rotirea unui pătrat după cum este indicat în Figura 1.3. Exemplul următor indică de asemenea cum se poate controla un dispozitiv de intrare și activa/dezactiva funcție de așteptare (idle function). În acest exemplu butoanele mouse-ului opresc sau pornesc rotirea pătratului.

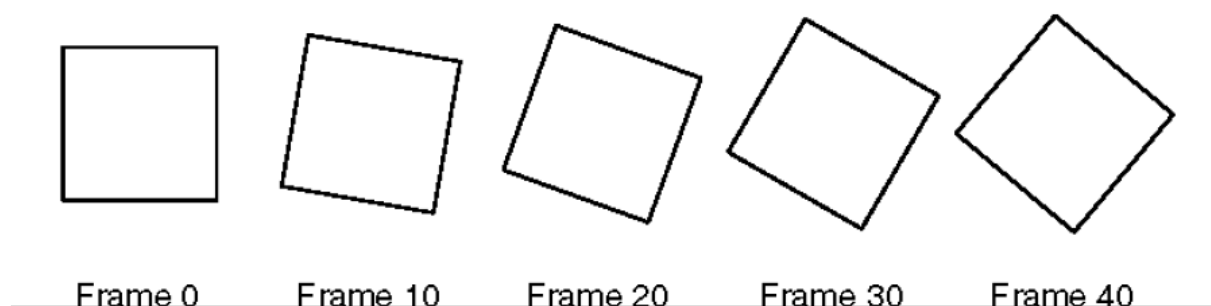


Figura 1.3. Rotirea unui pătrat folosind tehnica de dublu buffer

Exemplul 1.3. Program ce folosește tehnica dublu buffer: double.c

```
#include <windows.h>
#include <GL/gl.h>
#include <GL/glu.h>
#include <GL/glut.h>
#include <stdlib.h>

static GLfloat spin = 0.0;
void init(void)
{
    glClearColor (0.0, 0.0, 0.0, 0.0);
    glShadeModel (GL_FLAT);
}
void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPushMatrix();
    glRotatef(spin, 0.0, 0.0, 1.0);
    glColor3f(1.0, 1.0, 1.0);
    glRectf(-25.0, -25.0, 25.0, 25.0);
    glPopMatrix();
    glutSwapBuffers();
}
void spinDisplay(void)
{
    spin = spin + 2.0;
    if (spin > 360.0)
        spin = spin - 360.0;
    glutPostRedisplay();
}
void reshape(int w, int h)
{
    glViewport (0, 0, (GLsizei) w, (GLsizei) h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(-50.0, 50.0, -50.0, 50.0, -1.0, 1.0);
    glMatrixMode(GL_MODELVIEW);
```

```

        glLoadIdentity();
    }
    void mouse(int button, int state, int x, int y)
    {
        switch (button) {
            case GLUT_LEFT_BUTTON:
                if (state == GLUT_DOWN)
                    glutIdleFunc(spinDisplay);
                break;
            case GLUT_MIDDLE_BUTTON:
                if (state == GLUT_DOWN)
                    glutIdleFunc(NULL);
                break;
            default:
                break;
        }
    }
}
/*
 * Initializarea modului de afisare dublu-buffer
 * Inregistrarea functiilor callback pentru tratarea
 * evenimentelor generate de mouse
 */
int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode (GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize (250, 250);
    glutInitWindowPosition (100, 100);
    glutCreateWindow (argv[0]);
    init ();
    glutDisplayFunc(display);
    glutReshapeFunc(reshape);
    glutMouseFunc(mouse);
    glutMainLoop();
    return 0;
}

```

2. Managementul stărilor și desenarea obiectelor geometrice

Obiective ale capitolului

În acest capitol ne propunem să cunoaștem următoarele:

- Ștergerea ferestrei cu o culoare arbitrară
- Forțarea oricărei operații de desenare, aflată în așteptare, să se execute
- Desenarea cu orice primitivă geometrică – puncte, linii și poligoane – în două sau trei dimensiuni
- Activarea/Dezactivarea și interogarea variabilelor de stare
- Controlarea modului de afișare a primitivelor grafice (ex. linii întrerupte, etc...)
- Specificarea normalelor la suprafața unor corpuri solide prin intermediul unor puncte potrivite
- Folosire vectorilor de vertex-uri pentru stocarea și accesarea datelor geometrice doar cu câteva apeluri de funcții
- Salvarea și restaurarea stărilor mai multor variabile de stare în același timp

Cu toate că se pot desena imagini interesante și complexe folosind OpenGL, toate aceste obiecte sunt construite dintr-un număr mic de primitive de bază.

La cel mai înalt nivel de abstractizare, există trei operații de bază pentru a desena o scenă: ștergerea ferestrei, desenarea unui obiect geometric și desenarea unui obiect raster (rasterizare - conversia unei scene 3D realizată din poligoane și scrisă în buffer-ul de cadre, într-o imagine completă cu texturi, adâncimi și iluminare).

Acest capitolul este împărțit în secțiunile următoarele:

- **„Scurtă introducere în operațiile de bază pentru desenare”** explică cum se poate șterge o fereastră și cum se poate forța executarea operațiilor de desenare pentru a obține o scenă completă. Oferă de asemenea informații de bază pentru controlarea culorilor atribuite obiectelor grafice și mai descrie sistemul de coordonate folosit în OpenGL.
- **„Descrierea punctelor, liniilor și poligoanelor”** indică ce este setul de primitive grafice și cum poate fi folosit.
- **„Managementul de bază al variabilelor de stare”** descrie cum se pot activa-dezactiva sau interoga variabilele de stare.

- **„Afișarea punctelor, liniilor și poligoanelor”** explică ce control are programatorul asupra detaliilor legate de modul de desenare a primitivelor (ex ce diametru au punctele, dacă liniile sunt continue sau întrerupte, dacă poligoanele au margini sau sunt umplute, etc...).

2.1. Scurtă introducere în operațiile de bază pentru desenare

Acest subcapitol explică cum se poate șterge o fereastră în scopul de a o folosi pentru desenare, de a seta culorile obiectelor ce vor fi desenate și de a forța ca procesul de desenare să fie complet. Nici unul dintre aceste subiecte nu au vreo legătură cu obiectele geometrice într-un mod direct, dar orice program care desenează obiecte geometrice trebuie mai devreme sau mai târziu să trateze aceste probleme.

2.1.1 Ștergerea ferestrei

Desenarea pe ecranul unui monitor este diferită față de desenarea pe o foaie de hârtie deoarece foaia de hârtie este la început albă și tot ceea ce este de făcut este să desenezi. Într-un calculator, memoria ce conține imaginea este de obicei umplută cu ultima imagine ce a fost desenată, astfel încât este nevoie să ștergem memoria cu o culoare de fundal înainte de a începe desenarea unei noi imagini. Culoarea care se folosește pentru fundal depinde de aplicație. De exemplu: pentru un procesor de cuvinte (Word, Excel) culoarea fundalului ar putea fi albă ca și culoarea foii. Dacă se va desena o vedere din interiorul unei nave spațiale culoarea de fundal va fi culoarea neagră și se vor desena apoi stele, planete, etc. Câteodată s-ar putea să nu fie nevoie să ștergem ecranul deloc. De exemplu: dacă imaginea reprezintă interiorul unei camere, atunci întreaga fereastră este acoperită în momentul când se desenează pereții.

Înainte de a șterge fereastra trebuie cunoscut cum sunt stocate culorile pixelilor în hărțile de biți. Există două moduri de stocare: ori se stochează în hărțile de biți valorile de verde, galben, albastru sau alfa ale unui pixel (RGBA - red, green, blue, alfa), ori se folosește un index care referă o valoare dintr-un tabel de culori (lookup table). Modul de afișare al culorilor RGBA este cel mai frecvent folosit astfel încât îl vom folosi și noi în cele mai multe exemple din această lucrare. Pentru moment se pot ignora toate referințele la valoarea alfa.

De exemplu: următoarele linii de cod șterg o fereastră setată în modul RGBA la culoarea neagră:

```
glClearColor(0.0, 0.0, 0.0, 0.0);
glClear(GL_COLOR_BUFFER_BIT);
```

Prima linie setează culoarea de ștergere la culoarea neagră, iar a doua linie șterge fereastra cu culoarea de ștergere. Parametrul de la funcția **glClearColor()** indică care buffer trebuie șters. În acest caz, programul șterge doar buffer-ul de culori, unde este păstrată imaginea afișată pe ecran. De obicei, se setează culoarea de ștergere o singură dată la începutul execuției programului și mai apoi se pot șterge buffer-ele de câte ori este nevoie. OpenGL reține valoarea culorii de ștergere într-o variabilă de stare, astfel încât nu mai este nevoie să fie setată de câte ori se șterge un buffer.

De exemplu: pentru a șterge buffer-ul de culori și buffer-ul de adâncime se poate folosi următoarea secvență de comenzi:

```
glClearColor(0.0, 0.0, 0.0, 0.0);  
glClearDepth(1.0);  
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

În acest caz apelul funcției **glClearColor()** realizează același lucru ca și în exemplu anterior, comanda **glClearDepth()** specifică valoarea cu care fiecare pixel din buffer-ul de adâncime ar trebui setat și parametrul comenzii **glClear()** este alcătuită acum din buffer-ele care trebuiesc șterse unite printr-un SAU pe biți.

```
void glClearColor(GLclampf red, GLclampf green, GLclampf blue, GLclampf alpha);
```

Setează culoarea de ștergere folosită pentru a șterge buffer-ele de culoare în modul de afișare RGBA. Dacă este necesar, valorile roșu, albastru, verde și alfa sunt trunchiate la [0,1]. Valoarea de ștergere predefinită este (0,0,0,0), valori ce reprezintă culoarea neagră.

```
void glClear(GLbitfield mask);
```

Șterge buffer-ele specificate cu valorile de ștergere curente. Argumentul **mask** reprezintă o combinație de SAU pe biți a valorilor din Tabelul 2.1.

Buffer	Nume
Buffer de culoare	GL_COLOR_BUFFER_BIT
Buffer de adâncime	GL_DEPTH_BUFFER_BIT
Buffer de acumulare	GL_ACCUM_BUFFER_BIT
Buffer de șabloane	GL_STENCIL_BUFFER_BIT

Tabel 2.1 Buffere ce pot fi șterse

Înainte de a apela comanda de ștergere a buffere-lor, trebuie setată valoarea de ștergere a buffere-lor dacă se doresc alte valori diferite de cele predefinite în sistemul OpenGL. În plus, pe lângă comenzile **glClearColor()** și **glClearDepth()** mai există și **glClearIndex()**, **glClearAccum()** și **glClearStencil()** folosite pentru a specifica valoarea indexului de culoare, culoarea de acumulare și indexul șablonului folosite la ștergerea buffere-lor corespondente.

OpenGL permite ștergerea în același timp a mai multor buffere, deoarece operația de ștergere a unui buffer este de obicei lentă datorită faptului că fiecare pixel din fereastră trebuie procesat și unele plăci grafice oferă posibilitatea ștergerii mai multor buffere în același timp. Dispozitivele ce nu permit această operație, folosesc o ștergere secvențială. Deși următoarele linii de cod

```
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);  
și  
glClear(GL_COLOR_BUFFER_BIT);  
glClear(GL_DEPTH_BUFFER_BIT);
```

au același rezultat, diferența dintre ele este că primul exemplu are o viteză de execuție mai mare decât al doilea.

2.1.2 Specificarea unei culori

Cu OpenGL, descrierea formei unui obiect desenat este independentă de descrierea culorilor atribuite lui. De fiecare dată când un obiect geometric este desenat, acest lucru se realizează folosind schema de culori curentă. Schema de culori poate fi la fel de simplă ca și cum ai desena totul cu culoarea roșie, sau poate fi mai complicată când dorim desenarea unui obiect format din plastic albastru, pe care există spoturi de lumină galbenă și mai există o luminozitate ambientală roșiatică. În general, un programator OpenGL, setează mai întâi culoarea sau schema de culori și apoi desenează obiectele. Până când culoarea sau schema de culori este modificată, toate obiectele sunt desenate folosind acea culoare sau schemă de culori. Folosind această metodă, orice aplicație OpenGL va avea o performanță mai bună, deoarece OpenGL reține valorile curente pentru ștergere.

De exemplu, pseudocodul următor:

```
set_current_color(red);  
draw_object(A);  
draw_object(B);  
set_current_color(green);  
set_current_color(blue);  
draw_object(C);
```

Desenează obiectele A și B în roșu și obiectul C în albastru. Comanda de pe linia a patra care setează culoarea curentă de desenare la verde nu mai are nici un efect.

Pentru a seta o culoare se folosește comanda **glColor3f()**, comandă ce primește trei parametri ce sunt de tip flotant (float) între 0.0 și 1.0. Parametrii sunt roșu, verde și albastru în această ordine și descrie culoarea în format RGB. Aceste valori combinate formează culoarea finală, unde o valoare de 0.0 indică absența respectivei culori, iar o valoare de 1.0 indică folosirea culorii la intensitate maximă.

De exemplu, comanda

```
glColor3f(1.0, 0.0, 0.0);
```

desenează obiectele cu culoarea roșie la intensitate maximă, fără nici una dintre culorile verde sau albastru. Toate argumentele setate la valoarea 0.0 creează culoarea neagră și în contrast toate setate la valoarea 1.0 creează culoarea albă. Setând toate argumentele la valoarea 0.5 creează o culoare gri. În continuare se oferă o serie de exemple:

```
glColor3f(0.0, 0.0, 0.0);    negru
glColor3f(1.0, 0.0, 0.0);    roșu
glColor3f(0.0, 1.0, 0.0);    verde
glColor3f(1.0, 1.0, 0.0);    galben
glColor3f(0.0, 0.0, 1.0);    albastru
glColor3f(1.0, 0.0, 1.0);    magenta
glColor3f(0.0, 1.0, 1.0);    cyan
glColor3f(1.0, 1.0, 1.0);    alb
```

2.1.3 Forțarea terminării procesului de desenare

Cele mai moderne sisteme grafice sunt gândite ca o linie de asamblare. Fără intervenții ale dispozitivelor hardware, procesorul inițializează o comandă de desenare; apoi se realizează transformările geometrice; se realizează tăierea, urmată de umbrire și/sau texturizare. În final valorile sunt scrise în biți pentru afișare. În arhitecturile hardware performante fiecare dintre aceste operații se realizează de către o altă componentă hardware, creată special pentru a mări viteza de procesare. Într-o astfel de arhitectură nu este nevoie ca procesorul să aștepte terminarea unei comenzi de desenare ca să poată trece la următoarea operație. În timp ce procesorul se ocupă de un vertex din pipeline, dispozitivele de transformare lucrează să transforme ultimul vertex transmis, cel anterior este tăiat, ș.a.m.d. Dacă procesorul ar aștepta după fiecare comandă să își termine execuția, ca să poată trece la următoarea, viteza de execuție ar fi încetinită enorm și nu s-ar mai pune în discuție performanța sistemului.

Pe de altă parte, aplicația ar putea rula pe mai mult decât o mașină. De exemplu, presupunem că programul principal rulează pe o mașină numită client și programele care văd rezultatele desenării sunt pe mașina numită server sau terminal, care este conectată prin rețea la mașina client. Intr-un astfel de caz, ar fi total inefficient ca fiecare comandă să fie transmisă în același timp și prin rețea. De obicei, clientul adună o colecție de comenzi într-un singur pachet înainte de al trimite la server. Din păcate, codul de rețea de pe client nu are nici o posibilitate de a afla dacă programul grafic a terminat de desenat un frame sau o scenă. În cel mai rău caz, așteaptă la infinit pentru destule comenzi adiționale pentru a completa un pachet, și astfel nu mai avem acces niciodată la afișarea desenului complet.

Din acest motiv, OpenGL pune la dispoziție comanda **glFlush()**, care forțează clientul să trimită pachete prin rețea către server, chiar dacă acesta s-ar putea să nu fie complet. Unde nu se comunică prin rețea, toate comenzile sunt executate imediat pe server și comanda **glFlush()** nu mai are nici un efect. Totuși dacă scriem un program care vrem să funcționeze corect atât cu sau fără comunicare prin rețea trebuie inclusă apelarea comenzii **glFlush()** la sfârșitul fiecărui frame sau scene. **glFlush()** nu așteaptă ca un desen să fie realizat complet, ci doar forțează ca execuția desenului să înceapă.

Mai sunt alte situații în care comanda **glFlush()** este utilă:

- Pentru aplicații de randare ce creează imagini în memoria sistemului și nu se dorește ca ecranul să fie actualizat constant.
- Pentru implementări care adună un set de comenzi de randare utilizate pentru amortizarea necesităților de startare. Exemplul de mai sus, cu transmisia prin rețea este unul dintre astfel de cazuri.

Sintaxa:

```
void glFlush(void);
```

Forțează comenzile OpenGL apelate anterior, să-și înceapă execuția, garantând realizarea acestora în timp finit.

Dacă funcția **glFlush()** nu este suficientă, se poate folosi comanda **glFinish()**. Acesta are același efect ca și comanda **glFlush()**, doar că apoi așteaptă o notificare de la dispozitivul grafic sau de la rețea, care indică dacă desenul este complet în buffer-ul de frame-uri. Dacă dorim să sincronizăm mai multe task-uri ar fi bine să utilizăm **glFinish()**. De exemplu: în cazul când dorim să fim siguri că randarea tri-dimensională este pe ecran înainte să utilizăm Display PostScript pentru a edita etichete deasupra desenului. Un alt exemplu ar fi să ne asigurăm că desenul este complet afișat înainte se a accepta date de intrare de la utilizator. După ce se apelează comanda **glFinish()** firul de execuție al programului este blocat până

când se primesc date de la placa grafică precum că desenul a fost complet afișat. Rețineți totuși că folosirea excesivă a comenzii **glFinish()** poate reduce performanța aplicației mai ales dacă se rulează pe o rețea. Este recomandat să se folosească comanda **glFlush()**.

Sintaxa:

```
void glFinish(void);
```

Forțază toate comenzile OpenGL apelate anterior să-și termine execuția. Această comandă nu returnează nimic până când toate comenzile anterioare sunt executate cu succes.

2.1.4 Sistemele de coordonate

De fiecare dată când se deschide o fereastră sau atunci când fereastra este redimensionată sau deplasată, atunci sistemul de operare va trimite un mesaj de notificare. Dacă se folosește GLUT, notificarea se face automat; atunci când se generează un astfel de mesaj, programul va apela funcția callback înregistrată de comanda **glutReshapeFunc()**. Trebuie înregistrată o funcție callback care va

- restabili regiunea dreptunghiulară care va fi noua suprafață de desenare și randare
- defini noul sistem de coordonate folosit la desenarea obiectelor

În capitolul următor se va oferi o descriere mai amănunțită a sistemelor de coordonate tridimensionale, dar momentan vom folosi doar un sistem de coordonate bidimensional care îl vom folosi la desenarea câtorva obiecte. Exemplul 2.1 înregistrează funcția callback **reshape()** cu comanda **glutReshapeFunc(reshape)**:

Exemplu 2.1 Funcția callback **reshape()**

```
void reshape (int w, int h)
{
    glViewport (0, 0, (GLsizei) w, (GLsizei) h);
    glMatrixMode (GL_PROJECTION);
    glLoadIdentity ();
    gluOrtho2D (0.0, (GLdouble) w, 0.0, (GLdouble) h);
}
```

La generarea evenimentului, GLUT va apela această funcție cu două argumente: lungime și înălțime, valori date în pixeli ce indică noua dimensiune sau poziție a ferestrei. Comanda **glViewport()** ajustează suprafața de desenare la noua dimensiune a ferestrei. Următoarele trei rutine setează sistemul de coordonate pentru desenare astfel încât colțul din stânga-jos să aibă coordonatele (0,0), și colțul din dreapta-sus să fie (w, h).

A se vedea Figura 2.1.

Pentru a ilustra mai bine conceptul, să ne imaginăm o bucată de hârtie milimetrică. Argumente w și h din funcția **reshape()** reprezintă numărul de pătrățele formate din coloanele și liniile de pe hârtia milimetrică. Mai apoi trebuie fixat un sistem de axe. Comanda **glutOrtho2D()** fixează originea la $(0,0)$, și face ca fiecare pătrat să reprezinte o unitate. Acum, când se vor randa puncte, linii sau poligoane, primitivele vor apărea în pătrate ușor de determinat.

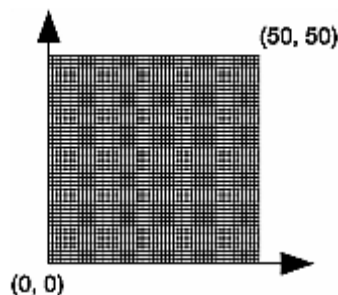


Figura 2.1 Sistem de coordonate definit pentru $w=50$ și $h=50$

2.2 Descrierea punctelor, liniilor și poligoanelor

Acest subcapitol explică cum pot fi create și desenate primitivele geometrice din OpenGL. Toate primitivele sunt descrise prin intermediul vertex-urilor (coordonate ce definesc puncte, capetele unui segment sau colțurile unui poligon).

2.2.1 Ce sunt punctele, liniile și poligoanele în OpenGL?

Probabil oricine are o idee despre semnificația matematică a acestor termeni, însă cu toate că semnificația pe care o atribuie OpenGL este similară, ea nu este aceeași.

O diferență apare datorită limitărilor calculelor realizate de calculator. În orice implementare OpenGL, calculele în virgulă mobilă au o precizie limitată și prezintă erori la rotunjire. Astfel, și coordonatele OpenGL ale punctelor, liniilor și poligoanelor suferă de aceleași neajunsuri.

O altă diferență importantă apare din limitările dispozitivului de afișare. Pe un astfel de ecran cea mai mică unitate afișabilă este pixelul și cu toate că pixelii pot fi mai mici decât $1/100$ dintr-un inch, sunt totuși cu mult mai mari decât conceptele matematice de infinit de mic (pentru puncte) sau infinit de subțiri (pentru linii). Când OpenGL realizează calcule, presupune că punctele sunt reprezentate de vectori de numere raționale. Cu toate acestea un punct este de obicei (dar nu întotdeauna) desenat ca un singur pixel și multe alte puncte cu coordonate foarte puțin diferite ar putea fi desenate de OpenGL cu același pixel.

Puncte

Un punct este reprezentat de un set de numere raționale ce poartă numele de vertex. Toate calculele interne sunt realizate ca și cum vertex-urile ar fi tridimensionale. Vertex-urile specificate de utilizator ca fiind bidimensionale (adică au numai coordonatele x și y), în realitate au și coordonate z , numai că OpenGL le setează la 0.

OpenGL funcționează în coordonatele omogene ale geometriei tridimensionale, astfel încât calculele interne toate vertex-urile sunt reprezentate cu patru coordonate raționale (x , y , z , w). Dacă w este diferit de 0, aceste coordonate corespund punctului tridimensional Euclidean (x/w , y/w , z/w). Se poate specifica coordonata w în comenzile OpenGL dar este rareori folosită. Dacă coordonata w nu este specificată, aceasta se subînțelege a fi 1.0.

Linii

În OpenGL termenul de linie se referă la un segment de linie și nu la versiunea matematică a liniei care o extinde la infinitate în ambele direcții. Există modalități ușoare de a specifica o serie de segmente conectate sau chiar o buclă închisă. În toate cazurile liniile ce formează seriile conectate sunt specificate în termeni de vertex-uri ce definesc capetele segmentelor.

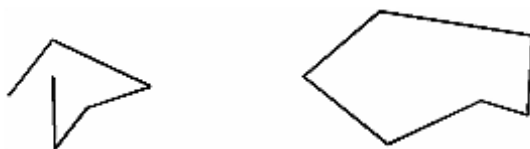


Figura 2.2 Două serii de segmente conectate

Poligoane

Poligoanele sunt suprafețe închise de o singură buclă închisă a unei linii segment, care este descrisă de vertice prin capetele sale. Poligoanele sunt de obicei desenate cu interiorul umplut, dar se pot desena doar trasând liniile exterioare sau ca un set de puncte.

În general, poligoanele pot fi complicate, astfel încât OpenGL impune restricții în ceea ce presupune primitiva poligon.

- Primul lucru impus este faptul că marginile poligoanelor din OpenGL nu se pot intersecta.
- Al doilea lucru impus este faptul că poligoanele din OpenGL sunt convexe.

O suprafață este convexă dacă linia care unește oricare două puncte din interiorul suprafeței se află tot în interiorul acesteia. Ca exemplificare, se poate observa în Figura 2.3, câteva poligoane valide și invalide în OpenGL.

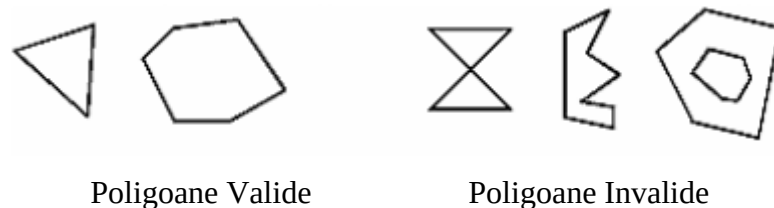


Figura 2.3 Poligoane valide și invalide

După cum se observă în Figura 2.3, OpenGL nu restricționează însă, numărul de linii segment care formează marginea exterioară a unui poligon convex, dar poligoanele cu găuri de asemenea nu pot fi descrise.

Motivul pentru care OpenGL-ul a restricționat tipul de poligoane este pentru creșterea vitezei plăcii video pentru procesul de randare a desenelor. Logic, poligoanele simple sunt randate mult mai rapid. Poligoanele mai complexe sunt mai greu de procesat rapid. Astfel, pentru o performanță maximă OpenGL presupune poligoanele ca fiind doar poligoane simple.

Suprafețele din lumea reală sunt de obicei poligoane complexe, neconvexe sau cu găuri. Dar cum aceste poligoane sunt formate prin unirea de poligoane simple, obiectele complexe pot fi create și cu OpenGL prin apelarea unor funcții din Librăria GLU. Acestea sunt realizate pentru a prelucra poligoane complexe și a oferi o descriere a acestora în grupuri de poligoane simple, ușor și rapid de renderizat de OpenGL.

Știind că vertex-urile în OpenGL sunt întotdeauna tridimensionale, punctele care formează marginile unui poligon particular, nu este necesar să fie în același plan. Dacă întâlnim acest caz, după mai multe rotații în spațiu, schimbări a unghiului de unde se privește desenul și proiecții pentru afișarea pe ecran, este foarte posibil ca punctele să nu mai formeze un poligon convex simplu. De exemplu, dacă urmărim transformările din Figura 2.4, putem observa cum un poligon care nu are toate puncte în același plan s-a transformat pentru randare într-un poligon care nu mai este simplu. Intr-un astfel de caz nu se poate garanta că randarea se va realiza complet. De aceea este de preferat să utilizăm triunghiuri la descrierea poligoanelor, deoarece întotdeauna trei puncte vor fi în același plan.

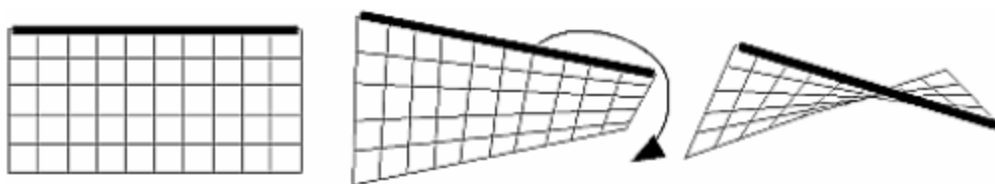


Figura 2.4 Poligon neplanar transformat într-un poligon care nu mai este simplu.

2.2.2 Dreptunghiuri

Dreptunghiurile sunt foarte des întâlnite în aplicațiile grafice și datorită acestui lucru, OpenGL pune la dispoziție o primitivă **glRect*()**. În OpenGL se poate desena un dreptunghi ca și un poligon, însă există această implementare optimizată specifică doar pentru dreptunghiuri.

Sintaxa:

```
void glColor{sfid}(TYPEx1, TYPEy1, TYPEx2, TYPEy2);  
void glColor{sfid}v(TYPE*v1, TYPE*v2);
```

Desenează dreptunghiul definit de colțurile aflate în punctele (x1,y1) și de (x2,y2). Dreptunghiul se află în planul $z=0$ și este paralel cu axele x și y. Dacă vectorul din funcție este utilizat, colțurile sunt specificate prin doi pointeri spre un array, fiecare conținând o pereche (x, y).

Deși toate dreptunghiurile încep cu o orientare în spațiul tridimensional (în planul x-z și paralel cu axele), se poate schimba acest lucru prin aplicarea de rotații sau alte transformări.

2.2.3 Curbe și suprafețe curbate

Toate liniile ușor curbate pot fi approximate, cu un oarecare grad de precizie ales arbitrar, printr-o linie scurtă de segmente sau mici suprafețe poligonale. De exemplu în Figura 2.5, divizăm liniile și suprafețele curbe suficient și apoi le aproximăm cu o linie dreaptă de segmente sau cu poligoane plate, până când acestea apar curbe.



Figura 2.5 Aproximarea curbilor

2.2.4 Specificarea Vertex-urilor

În OpenGL toate obiectele geometrice sunt descrise ca un set ordonat de vertex-uri. Pentru a specifica un vertex se utilizează comanda **glVertex*()**.

Sintaxa:

```
void glVertex{234}{sfid}[v](TYPE coords);
```

Specifică un vertex folosit pentru a descrie un obiect geometric. Se pot specifica până la patru coordonate (x, y, z, w) pentru un vertex particular, însă un vertex trebuie specificat prin

cel puțin două coordonate (x, y) (se selectează comanda cea mai apropiată de ceea ce se dorește). Dacă se folosește un vertex unde nu se specifică coordonatele z sau w, se înțelege că z este 0, iar w ca fiind 1. Apelarea comenzii **glVertex*()** are efect doar dacă este apelată între o pereche de comenzi **glBegin()** și **glEnd()**.

Exemplul 2.2 Utilizarea comenzii **glVertex*()**

```
glVertex2s(2, 3);  
glVertex3d(0.0, 0.0, 3.1415926535898);  
glVertex4f(2.3, 1.0, -2.2, 2.0);  
  
GLdouble dvect[3] = {5.0, 9.0, 1992.0};  
glVertex3fv(dvect);
```

Primul exemplu reprezintă un vertex cu coordonate tridimensionale (2, 3, 0). (De amintit că dacă nu se menționează coordonata z, aceasta se consideră 0. Coordonatele în al doilea exemplu sunt (0.0, 0.0, 3.1415926535898)(sunt scrise în numere flotante cu dublă precizie).

Exemplul al treilea reprezintă vertex-ul cu coordonate tridimensionale (1.15, 0.5, -1.1), aici intervine situația când coordonatele x, y, z se divid cu valoarea coordonatei w. În ultimul exemplu **dvect** este un pointer spre un array de trei numere flotante cu dublă precizie.

Pe unele calculatoare, vectorul folosit de comanda **glVertex*()** este mai eficient, deoarece în această situație doar un singur parametru este trimis subsistemului grafic. Există dispozitive hardware speciale ce pot să proceseze mai multe serii de coordonate ca și o singură unitate. Dacă avem o astfel de placă este în avantajul nostru să ordonăm datele în așa fel încât coordonatele vertex-urilor să fie împachetate secvențial în memorie. Se poate astfel câștiga în performanță, utilizând operațiile pe vertex-uri sub formă de array-uri.

2.2.5 Primitivele geometrice de desenare în OpenGL

După specificarea vertex-urilor trebuie să aflăm cum creăm în OpenGL un set de puncte, o linie, un poligon din aceste vertex-uri. Astfel, mai întâi, împărțim seturile de vertex-uri între o secvență care începe cu apelarea comenzii **glBegin()** și se termină cu apelarea comenzii **glEnd()**. Argumentul transmis de **glBegin()** determină de ce tip este primitiva geometrică care se va construi din vertex-uri. De exemplu, Exemplul 2.3 specifică vertex-urile pentru poligonul din Figura 2.6.

Exemplul 2.3 Poligon umplut

```
glBegin(GL_POLYGON);  
    glVertex2f(0.0, 0.0);  
    glVertex2f(0.0, 3.0);  
    glVertex2f(4.0, 3.0);  
    glVertex2f(6.0, 1.5);  
    glVertex2f(4.0, 0.0);  
glEnd();
```

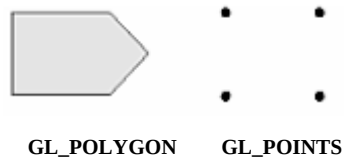


Figura 2.6 Desenarea unui poligon sau a unui set de puncte

Dacă am utilizat `GL_POINTS`, în loc de `GL_POLYGON`, desenul nu va avea decât cele 5 puncte ca în Figura 2.6.

În Tabelul 2.2 se va prezenta lista cu 10 argumente posibile folosite de **glBegin()** și tipul de primitive corespondente.

Sintaxa:

```
void glBegin(GLenum mode);
```

Marchează începutul unei liste de date a vertex-urilor care descriu o primitivă geometrică.

Tipul primitivei este indicat de `mod`, care poate fi unul din valorile din Tabelul 2.2.

Tabelul 2.2: Numele și semnificația primitivelor geometrice

Valoare	Semnificație
<code>GL_POINTS</code>	puncte individuale
<code>GL_LINES</code>	perechi de vertice interpretate ca linii segment individuale
<code>GL_LINE_STRIP</code>	serii de linii segment conectate
<code>GL_LINE_LOOP</code>	la fel ca mai sus, cu un segment adăugat între ultimul și primul vertex
<code>GL_TRIANGLES</code>	tripleți de vertice interpretate ca triunghiuri
<code>GL_TRIANGLE_STRIP</code>	triunghiuri cu o latură comună
<code>GL_TRIANGLE_FAN</code>	triunghiuri cu o latură comună în formă de evantai
<code>GL_QUADS</code>	cvadrupleți de vertex-uri interpretate ca poligoane cu patru laturi
<code>GL_QUAD_STRIP</code>	cvadrupleți cu laturi comune
<code>GL_POLYGON</code>	limita unui poligon simplu convex

Sintaxa:

```
void glEnd(void);
```

Marchează sfârșitul liste de date a unui vertex.

În Figura 2.7 se prezintă câteva exemple ale primitivelor geometrice listate în Tabelul 2.2.

Paragrafele ce urmează Figura 2.7 descriu pixelii care sunt desenați pentru fiecare obiect în parte. Unele tipuri de linii și poligoane sunt deja definite. Există mai multe posibilități de a desena aceeași primitivă; metoda care se alege depinde de datele vertex-ului. Se presupune că n vertex-uri ($v_0, v_1, v_2, \dots, v_{n-1}$) sunt descrise între perechea **glBegin()** și **glEnd()**.

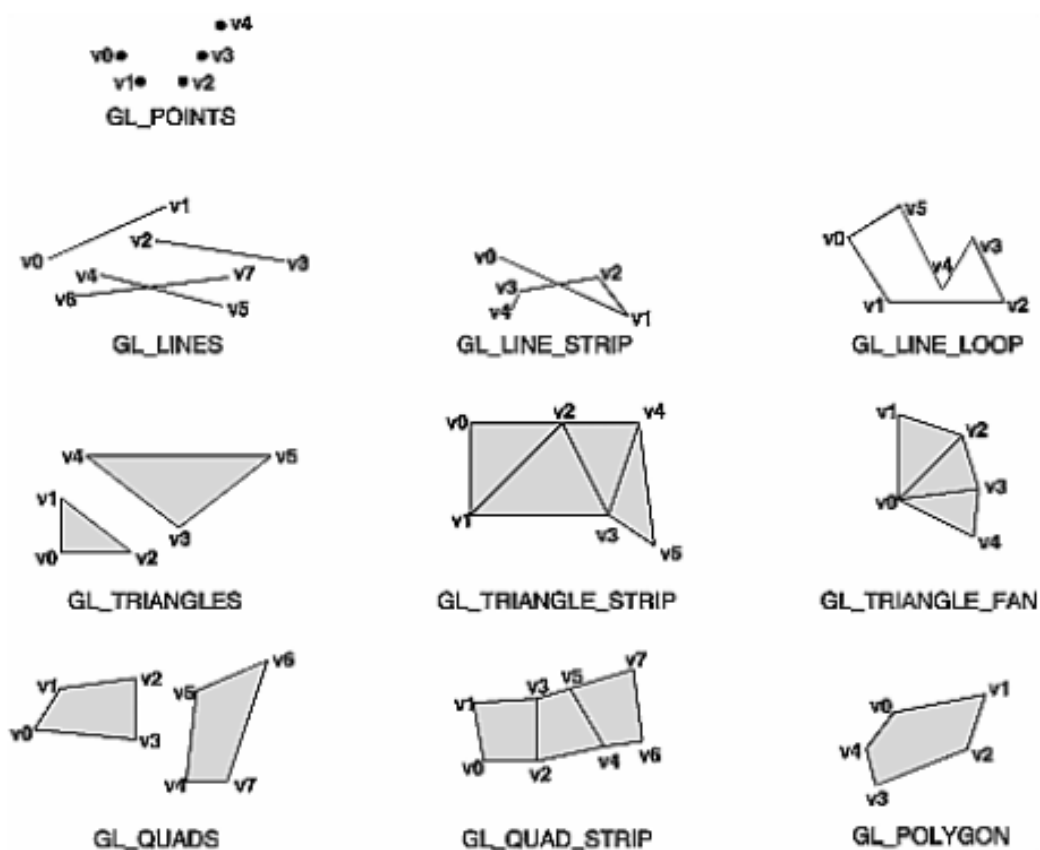


Figura 2.7 Tipuri de primitive geometrice

GL_POINTS - Desenează un punct în fiecare din cele n vertex-uri.

GL_LINES - Desenează o serie de linii de segment neconectate între ele. Segmentele sunt desenate între v_0 și v_1 , între v_2 și v_3 , ș.a.m.d. Dacă n este impar, ultimul segment desenat este între v_{n-3} și v_{n-2} , iar v_{n-1} este ignorat.

GL_LINE_STRIP - Desenează o linie de segment de la v_0 la v_1 și tot așa, până la final când desenează segmentul de la v_{n-2} la v_{n-1} . În total sunt desenate $n-1$ linii de segment. Nimic nu

este desenat, în schimb, dacă n nu este mai mare ca 1. Nu sunt restricții la vertex-urile care descriu o linie întreruptă sau buclă; liniile se pot intersecta arbitrar.

GL_LINE_LOOP – Desenează la fel ca GL_LINE_STRIP, cu excepția faptului că la final se desenează o linie de segment de la v_{n-1} la v_0 , realizând o buclă.

GL_TRIANGLES – Desenează o serie de triunghiuri (poligoane cu trei laturi) utilizând vertex-urile v_0, v_1, v_2 , apoi v_3, v_4, v_5 ș.a.m.d. Dacă n nu este exact multiplu de 3, vertex-ul final (sau ultimele două) este(sunt) ignorat(e).

GL_TRIANGLE_STRIP – Desenează o serie de triunghiuri utilizând vertex-uri v_0, v_1, v_2 , apoi v_2, v_1, v_3 , apoi v_2, v_3, v_4 ș.a.m.d. Ordinea aceasta este utilizată să fie desenate cu aceeași orientare. Păstrarea orientării este importantă pentru anumite operații, precum tăierea. Pentru ca să se deseneze ceva, este necesar ca n să fie cel puțin 3.

GL_TRIANGLE_FAN – Desenează la fel ca și GL_TRIANGLE_STRIP, cu excepția că ordinea este următoarea: v_0, v_1, v_2 , apoi v_0, v_2, v_3 , apoi v_0, v_3, v_4 ș.a.m.d.

GL_QUADS – Desenează o serie de poligoane cu patru laturi utilizând vertex-urile v_0, v_1, v_2, v_3 , apoi v_4, v_5, v_6, v_7 , ș.a.m.d. Dacă n nu este multiplu de 4, ultimele 3(2 sau 1) vertex-uri sunt ignorate.

GL_QUAD_STRIP – Desenează o serie de poligoane cu patru laturi începând cu v_0, v_1, v_3, v_2 , apoi cu v_2, v_3, v_5, v_4 , apoi v_4, v_5, v_7, v_6 ș.a.m.d. Trebuie ca n să fie cel puțin 4, altfel nu se poate desena nimic. Dacă n este impar, vertex-ul final este ignorat.

GL_POLYGON – Desenează un poligon utilizând punctele $v_0, \dots, v_{n-1}$ ca și vertex-uri. Trebuie ca n să fie cel puțin 3, altfel nu se poate desena nimic. Poligonul specificat nu trebuie să se intersecteze și trebuie să fie convex. Dacă vertex-urile nu satisfac aceste condiții, rezultatele sunt impredictibile.

Restricții pentru folosirea comenzilor *glBegin()* și *glEnd()*

Informațiile cele mai importante despre un vertex sunt coordonatele acestuia, care sunt specificate de comanda **glVertex*()**. Se pot adăuga informații suplimentare pentru un vertex, cum ar fi culoarea, un vector normal, coordonatele texturii, sau orice combinație a acestora,

utilizând comenzi speciale. Doar câteva dintre comenzi sunt valide între perechea **glBegin()** și **glEnd()**. Tabelul 2.3 conține o listă cu astfel de comenzi valide.

Tabelul 2.3 Comenzi acceptate între **glBegin()** și **glEnd()**

Comanda	Scopul comenzii
glVertex*()	setează coordonatele vertex-urilor
glColor*()	setează culoarea curentă
glIndex*()	setează indexul culorii curente
glNormal*()	setează coordonatele vectorului normal
glTexCoord*()	setează coordonatele texturii
glEdgeFlag*()	controlează desenarea marginilor
glMaterial*()	setează proprietățile materialului
glArrayElement()	extrage datele array ale vertex-ului
glEvalCoord*(), glEvalPoint*()	generează coordonate
glCallList(), glCallLists()	execută lista(ele) de afișare

Alte comenzi OpenGL decât cele care sunt scrise în Tabelul 2.3 nu mai sunt valide între perechea **glBegin()** și **glEnd()**, iar apelarea lor poate genera erori. Unele comenzi, cum ar fi **glEnableClientState()** și **glVertexPointer()** când sunt apelate între **glBegin()** și **glEnd()** au un comportament nedefinit care nu generează neapărat erori, de aceea trebuie reținut ca astfel de cazuri să evităm să le utilizăm. O astfel de rutină este și **glX*()**.

Cu toate că există atâtea restricții, în OpenGL se pot include și construcții din alte limbaje de programare. De exemplu, putem observa desenarea unui cerc în Exemplul 2.4.

Exemplul 2.4: Alte construcții între comenzile **glBegin()** și **glEnd()**

```
#define PI 3.1415926535898
GLint circle_points = 100;
glBegin(GL_LINE_LOOP);
for (i = 0; i < circle_points; i++) {
    angle = 2*PI*i/circle_points;
    glVertex2f(cos(angle), sin(angle));
}
glEnd();
```

Notă: Acest exemplu nu desenează un cerc în modul cel mai eficient posibil, mai ales dacă se intenționează a fi folosită în mod repetat. Comenzile grafice folosite sunt în mod uzual foarte

rapide, dar acest cod calculează un unghi și apelează funcțiile **sin()** și **cos()** pentru fiecare vertex; și în plus mai există timpii de procesare pentru bucla *for*.

Dacă nu sunt compilate într-o listă de afișare, toate comenzile **glVertex*()** ar trebui să apară între comenzile **glBegin()** și **glEnd()** (dacă apar în altă parte decât între cele două comenzi, ele nu au nici un efect).

Cu toate că multe comenzi sunt acceptate între perechea **glBegin()** și **glEnd()**, vertex-urile sunt generate doar la apelul unei comenzi **glVertex*()**. În momentul în care este apelată comanda **glVertex*()**, OpenGL atribuie culoarea curentă la vertex-ul creat, coordonatele texturii, informațiile despre vectorul de normale și altele. Pentru exemplificare se oferă următorul exemplu. Primul punct este desenat cu culoarea roșie, iar al doilea și al treilea sunt albastre, ignorându-se comenzile de setare a culorii ce sunt în plus.

```
glBegin(GL_POINTS);
glColor3f(0.0, 1.0, 0.0); /* green */
glColor3f(1.0, 0.0, 0.0); /* red */
glVertex(...);
glColor3f(1.0, 1.0, 0.0); /* yellow */
glColor3f(0.0, 0.0, 1.0); /* blue */
glVertex(...);
glVertex(...);
glEnd();
```

Cu toate că se poate folosi orice combinație de comenzi dintre cele 24 de versiuni ale comenzii **glVertex*()** între comenzile **glBegin()** și **glEnd()**, în aplicațiile reale toate apelurile realizate la un moment dat tind să aibă aceeași formă. Dacă specificațiile despre datele ce descriu vertex-uri sunt consistente și repetitive (ex. **glColor*()**, **glVertex*()**, **glColor*()**, **glVertex*()**, ...), o aplicație și-ar putea îmbunătăți performanțele folosind vectori de vertex-uri.

2.3 Managementul de bază al variabilelor de stare

În secțiunea anterioară, s-a prezentat un exemplu al unei variabile de stare, culoarea curentă RGBA, și cum poate ea fi asociată cu o primitivă. OpenGL administrează multe stări și variabile de stare. Un obiect poate fi renderizat cu lumini, texturi, înlăturarea suprafețelor ascunse, ceață sau alte stări ce iar putea influența modul cum arată.

Inițial, majoritatea stărilor sunt inactive. Aceste stări ar putea costa mult (în termeni de performanță) dacă ar fi activate; de exemplu, dacă s-ar activa maparea de texturi cu siguranță

s-ar încetini procesul de renderizare al unei primitive. Totuși, calitatea imaginii s-ar îmbunătăți și ar arăta mult mai realist, datorită posibilităților grafice îmbunătățite.

Pentru a activa sau dezactiva multe dintre aceste stări, se pot folosi următoarele două comenzi:

```
void glEnable(GLenum cap);
```

```
void glDisable(GLenum cap);
```

glEnable() activează o stare, iar **glDisable()** o dezactivează. Există peste 40 de valori ce pot fi folosite ca argumente pentru cele două comenzi. Unele dintre ele sunt **GL_BLEND** (ce controlează amestecul valorilor RGBA), **GL_DEPTH_TEST** (controlează comparațiile adâncimilor și salvează informațiile în buffer-ul de adâncime), **GL_FOG** (controlează efectul de ceață), **GL_LINE_STIPPLE** (controlează tipurile de linii), **GL_LIGHTING** (controlează lumina dintr-o scenă), ș.a.m.d.

De asemenea se mai poate interoga și starea curentă a unei variabile de stare.

```
GLboolean glIsEnabled(GLenum capability)
```

Returnează **GL_TRUE** sau **GL_FALSE**, în funcție de starea curentă a variabilei de stare, adică dacă ea este activă sau nu.

Stările prezentate anterior ca exemple au doar două setări posibile: activ sau inactiv. Pe lângă acestea, există rutine OpenGL care setează valori pentru variabile de stare cu mult mai complicate. De exemplu, comanda **glColor3f()** setează trei valori, ce fac parte din starea **GL_CURRENT_COLOR**. Există cinci rutine de interogare folosite pentru a afla ce valori sunt setate în variabilele de stare:

```
void glGetBooleanv(GLenum pname, GLboolean *params);
```

```
void glGetIntegerv(GLenum pname, GLint *params);
```

```
void glGetFloatv(GLenum pname, GLfloat *params);
```

```
void glGetDoublev(GLenum pname, GLdouble *params);
```

```
void glGetPointerv(GLenum pname, GLvoid **params);
```

Comenzile de mai sus pot obține valori booleene, întregi, în virgulă mobilă, în dublă precizie sau pointeri din variabilele de stare. Argumentul **pname** este o constantă simbolică ce indică asupra cărei variabile de stare se face interogarea și **params** este un pointer la un vector de tipul de dată indicat și va conține valorile obținute din variabila de stare. De exemplu, pentru a obține culoarea RGBA curentă se poate folosi **glGetInteger(GL_CURRENT_COLOR,params)** sau **glGetFloatv(GL_CURRENT_COLOR,params)**.

În cazul în care se dorește a se returna valorile de un anumit tip de dată se va efectua automat o conversie a tipurilor de date.

2.4 Afișarea punctelor, liniilor și poligoanelor

În mod predefinit, un punct este reprezentat printr-un singur pixel, o linie este desenată cu o lățime de un pixel și este solidă, iar poligoanele sunt umplute. Următoarele paragrafe prezintă modalități de a schimba aceste moduri de afișare predefinite.

2.4.1 Detaliile punctului

Pentru a controla dimensiunea unui punct de pe ecran, se folosește **glPointSize()** căreia i se furnizează dimensiunea dorită ca și argument.

```
void glPointSize(GLfloat size);
```

Setează lățimea în pixeli a unui punct; dimensiunea trebuie să fie mai mare de 0.0 și are valoarea predefinită de 1.0.

De fapt colecția de pixeli de pe un ecran, ce este desenată pentru dimensiuni variate depinde de faptul dacă este activată antialiasing-ul sau nu. Dacă antialiasing-ul este dezactivat, valorile fracționare ale dimensiunilor sunt rotunjite la valori întregi și se desenează o regiune pătratică în locul unde sunt pixelii. Dacă dimensiunea este de 1.0, pătratul este de un pixel pe un pixel, dacă dimensiunea este de 2.0 atunci pătratul este de 2 pixeli pe 2 pixeli, ș.a.m.d.

Dacă antialiasing-ul este activat, se desenează un grup circular de pixeli, iar marginile sunt colorate mai puțin intens, așa încât acestea să nu pară colțuroase. Astfel nici o dimensiune nu este rotunjită la valori întregi.

2.4.2 Detaliile liniei

În OpenGL o linie se poate specifica cu grosimi diferite, iar acesta poate fi desenată sub mai multe forme, continuă, punctată, sau desenată alternativ prin puncte și linii mai mici.

2.4.2.1 Linii continue

```
void glLineWidth(GLfloat width);
```

Setează dimensiunea în pixeli pentru o linie. Dimensiunea trebuie să fie mai mare decât 0.0, iar predefinit este de 1.0.

Renderizarea de linii este afectată de modul antialiasing, în același mod ca și la puncte. Fără antialiasing dimensiuni de 1, 2 și 3 desenează linii de 1, 2 și 3 pixeli grosime. Cu antialiasing-ul activat, liniile nu au grosimi de dimensiuni întregi, deoarece pixelii de pe

margine sunt desenați cu culoare de intensitate mai mică. Se poate obține o valoare floating a punctului utilizând **GL_LINE_WIDTH_RANGE** cu **glGetFloatv()**.

Dimensiunea grosimei nu este măsurată perpendicular pe linie. Este măsurată pe direcția lui y, dacă valoarea este mai mică ca și 1.0, altfel este măsurată pe direcția lui x. Randarea unei linii antialiasing este exact echivalent cu randarea unui dreptunghi umplut cu grosimea dată, centrat exact pe o linie.

2.4.2.2 Linii punctate

Pentru a desena linii punctate, se utilizează comanda **glLineStipple()** pentru a defini o textura punctată, iar apoi se activează cu **glEnable()**.

```
glLineStipple(1, 0x3F07);
```

```
glEnable(GL_LINE_STIPPLE);
```

```
void glLineStipple(GLint factor, GLushort pattern);
```

Setează textura curentă pentru linii. Argumentul pentru textură este o serie de 16 biți de 0 și 1, care se repetă atât cât să se genereze o linie punctată. 1 înseamnă că se desenează pixeli, iar 0 că nu. Se poate utiliza un factor între 1 și 255, cu care se multiplică fiecare serie consecutivă de 1 și 0. Linia punctată trebuie să fie activată, la fel ca mai sus, dar și de asemenea trebuie să fie dezactivată folosind același argument cu **glDisable()**.

Exemplificăm prin textura 0x3F07 (adică, 0011111100000111 în binar). Se va desena o linie de 3 pixeli activi, apoi 5 inactivi, 6 activi și 2 activi. Dacă se folosește un factor de 2, textura va fi prelungită cu 6 pixeli activi, 10 inactivi, 12 activi, 4 inactivi. În Figura 2.8 se va descrie exemplul pentru mai multe texturi și mai mulți factori. Dacă linia punctată nu este activată, se va desena pe ecran linia cu textura 0xFFFF și factorul va fi 1. Se poate utiliza desenarea liniilor continue în combinație cu liniile punctate.

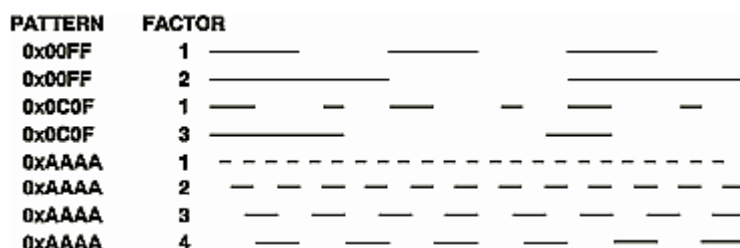


Figura 2.8 Linii punctate

După o altă abordare, este că textura liniei este shiftată cu câte un bit, odată cu desenarea unui pixel a acesteia. Când se desenează o singură dată, o serie de segmente de linie între **glBegin()** și **glEnd()**, textura continuă să se shifteze ca și cum un segment devine

următorul. În acest mod textura continuă printre o serie de segmente de linie conectate. Când **glEnd()** este executată, textura este resetată, și dacă mai multe linii sunt desenate înainte ca desenarea punctată să fie dezactivată, acesta se repornește de la începutul texturii. Dacă se desenează linii cu **GL_LINE**, textura se resetează pentru fiecare linie independentă.

Exemplul 2.5 ilustrează rezultatele desenării cu mai multe texturi și diferite grosimi ale liniei. Ilustrează de asemenea ce se întâmplă dacă se desenează segmente de linie individuale în loc de o singură linie punctată conectată. Rezultatul este ilustrat în Figura 2.9.



Figura 2.9: Linii continue punctate

Exemplul 2.5 Texturi de linii punctuate – lines.c

```
#include <windows.h>
#include <GL/gl.h>
#include <GL/glut.h>
#define drawOneLine(x1,y1,x2,y2); glBegin(GL_LINES); glVertex2f ((x1),(y1));
glVertex2f ((x2),(y2)); glEnd();
void init(void)
{
    glClearColor (0.0, 0.0, 0.0, 0.0);
    glShadeModel (GL_FLAT);
}
void display(void)
{
    int i;
    glClear (GL_COLOR_BUFFER_BIT);
    /* selectează culoarea albă pentru toate liniile */
    glColor3f (1.0, 1.0, 1.0);
    /* în primul rând, câte 3 linii, fiecare cu un alt tip de punctare */
    glEnable (GL_LINE_STIPPLE);
    glLineStipple (1, 0x0101); /* linie punctată */
    drawOneLine (50.0, 125.0, 150.0, 125.0);
    glLineStipple (1, 0x00FF); /* linie întreruptă */
    drawOneLine (150.0, 125.0, 250.0, 125.0);
```



```

    glLineStipple (1, 0x1C47); /* linie/ punct/ linie */
    drawOneLine (250.0, 125.0, 350.0, 125.0);
    /* în al 2-lea rând, 3 linii continue, fiecare cu un alt tip de punctare */
    glLineWidth (5.0);
    glLineStipple (1, 0x0101); /* punctat */
    drawOneLine (50.0, 100.0, 150.0, 100.0);
    glLineStipple (1, 0x00FF); /* întrerupt */
    drawOneLine (150.0, 100.0, 250.0, 100.0);
    glLineStipple (1, 0x1C47); /* linie/ punct/ linie */
    drawOneLine (250.0, 100.0, 350.0, 100.0);
    glLineWidth (1.0);
    /* în al 3-lea rând, 6 linii, cu linie/ punct/ linie */
    /* ca parte a unei singure linii punctate conectate */
    glLineStipple (1, 0x1C47); /* linie /punct / linie */
    glBegin (GL_LINE_STRIP);
    for (i = 0; i < 7; i++)
        glVertex2f (50.0 + ((GLfloat) i * 50.0), 75.0);
    glEnd ();
    /* în al 4-lea rând, 6 linii independente cu același tip de punctare */
    for (i = 0; i < 6; i++) {
        drawOneLine (50.0 + ((GLfloat) i * 50.0), 50.0,
            50.0 + ((GLfloat)(i+1) * 50.0), 50.0);
    }
    /* în al 5-lea rând, o linie, cu linie/ punct/ linie */
    /* cu un factor de 5 care repetă punctarea */
    glLineStipple (5, 0x1C47); /* linie/ punct/ linie */
    drawOneLine (50.0, 25.0, 350.0, 25.0);
    glDisable (GL_LINE_STIPPLE);
    glFlush ();
}

void reshape (int w, int h)
{
    glViewport (0, 0, (GLsizei) w, (GLsizei) h);
    glMatrixMode (GL_PROJECTION);
    glLoadIdentity ();
    gluOrtho2D (0.0, (GLdouble) w, 0.0, (GLdouble) h);
}

int main(int argc, char** argv)
{
    glutInit(&argc, argv);

```

```

glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
glutInitWindowSize (400, 150);
glutInitWindowPosition (100, 100);
glutCreateWindow (argv[0]);
init ();
glutDisplayFunc(display);
glutReshapeFunc(reshape);
glutMainLoop();
return 0;
}

```

2.4.3 Detaliile poligonului

Poligoanele sunt de obicei desenate prin umplerea unei regiuni create de unirea de linii, dar de asemenea și prin desenarea marginilor exterioare, cât și prin puncte și vertex-uri. Interiorul unui poligon poate fi desenat cu texturi, atât complete cât și punctate. În detaliu, poligoanele umplute sunt desenate în așa fel încât poligoanele adiacente să aibă o latură sau un vertex comun. Pixelii dintr-o astfel de latură sau vertex, sunt desenați o singură dată și sunt incluse doar într-un poligon. Nu se desenează de două ori, deoarece apar diferențe de culoare (mai închisă sau mai deschisă, în funcție de care poligon este transparent). Procesul de antialiasing pentru poligoane este mai complicat față de cel de la puncte sau linii.

2.4.3.1 Poligoane formate din puncte, linii exterioare sau umplute

Un poligon are două fețe – una frontală și cea din spate, și poate fi renderizat diferit în funcție din unghiul de unde privește observatorul. Acest lucru permite ca poligonului să i se taie părțile care nu sunt observabile. Predefinit, atât fața cât și spatele se desenează la fel. Pentru a schimba acest lucru, sau pentru a desena doar marginile exterioare sau vertex-urile, se folosește **glPolygonMode()**.

```
void glPolygonMode(GLenum face, GLenum mode);
```

Controlează modul de desenare a fețelor poligonului. Parametrul *face* poate fi **GL_FRONT_AND_BACK**, **GL_FRONT** sau **GL_BACK**; *mode* poate fi **GL_POINT**, **GL_LINE** sau **GL_FILL** indicând dacă modul de desenare va fi prin puncte, prin marginile exterioare sau umplut. Predefinit, ambele fețe sunt desenate ca fiind umplute.

De exemplu, pentru ca fețele la vedere să fie desenate umplut și cele din spate să fie desenate prin marginile exterioare se utilizează apelurile următoare:

```
glPolygonMode(GL_FRONT, GL_FILL);
```

```
glPolygonMode(GL_BACK, GL_LINE);
```

2.4.3.2 Inversarea și decuparea fețelor poligoanelor

Prin convenție, poligoanele a căror vertex-uri apar în ordine inversă sensului acelor de ceasornic sunt numite poligoane cu față frontală. Însă, se poate construi suprafața oricărui solid (rezonabil) din poligoane cu orientarea compatibilă, atât poligoane cu orientare invers sau în sensul acelor de ceasornic. (sferă, plăcintă, ceainic sunt orientabile; pe când sticlele Klein nu sunt orientabile).

Dacă presupunem că am descris un model al unei suprafețe orientabile, dar din întâmplare orientarea în sensul acelor de ceasornic este pe partea din spate, atunci în OpenGL putem să aflăm fața opusă, utilizând funcția **glFrontFace()**, adăugând orientarea dorită pentru poligoanele din față la vedere.

```
void glFrontFace(GLenum mode);
```

Controlează cum poligoanele cu față la vedere sunt determinate. Predefinit, *mode* este **GL_CCW**, ce corespunde unei orientări în sensul invers acelor de ceasornic, a vertex-urilor ordonate a unui poligon proiectat din coordonatele ferestrei. Dacă *mode* este **GL_CW**, fețele cu orientarea în sensul acelor de ceasornic sunt considerate fețe la vedere.

Intr-o suprafață complet închisă construită din poligoane opace cu o orientare compatibilă, nici unul din poligoanele din spate, nu vor fi vizibile niciodată – fiind acoperite de poligoanele din partea din față. Pentru a tăia poligoane din spate sau din față, se utilizează comanda **glCullFace()** și se activează tăierea cu **glEnable()**.

```
void glCullFace(GLenum mode);
```

Indică care poligoane ar trebui tăiate înainte de a fi convertite în coordonate pe ecran. Modul este **GL_FRONT**, **GL_BACK** sau **GL_FRONT_AND_BACK** pentru a indica partea din față, spate sau toate poligoanele. Pentru a avea efect tăierea trebuie să fie activată de **glEnable()** cu **GL_CULL_FACE**; poate fi dezactivată cu **glDisable()** folosind același argument.

În esență, pentru a exprima totul în termeni tehnici, decizia dacă fața unui poligon este la vedere sau se află în spate, depinde de semnul ariei poligonului calculată din coordonatele ferestrei.

2.4.3.3 Poligoane punctate (stippling)

Predefinit, poligoanele sunt desenate cu interiorul plin. Poligoanele pot fi desenate și cu modele definite în ferestre de 32x32 biți, ce se pot specifica cu comanda **glPolygonStipple()**.

```
void glPolygonStipple(const GLubyte *mask);
```

Definește modelul curent ce va fi folosit la umplerea poligoanelor. Argumentul **mask** este un pointer la un bitmap 32x32 ce este interpretată ca o mască de 0 și 1. Unde apare 1 pixelul corespunzător este desenat, iar unde apare 0 nu se desenează nimic.

Pe lângă definirea modelului curent de umplere a poligoanelor, mai trebuie activată variabila de stare pentru punctare:

```
glEnable(GL_POLYGON_STIPPLE);
```

Se folosește **glDisable()** cu același argument pentru a dezactiva punctarea.

Figura 2.11 indică rezultatele desenării unui poligon fără punctare și apoi folosind două modele diferite de punctare.

Programul este prezentat în Exemplul 2.6.

Inversa imaginii, adică alb pe negru, se obține desenând cu pixeli de culoare albă pe fundal negru folosind ca punctare modelul din Figura 2.10.

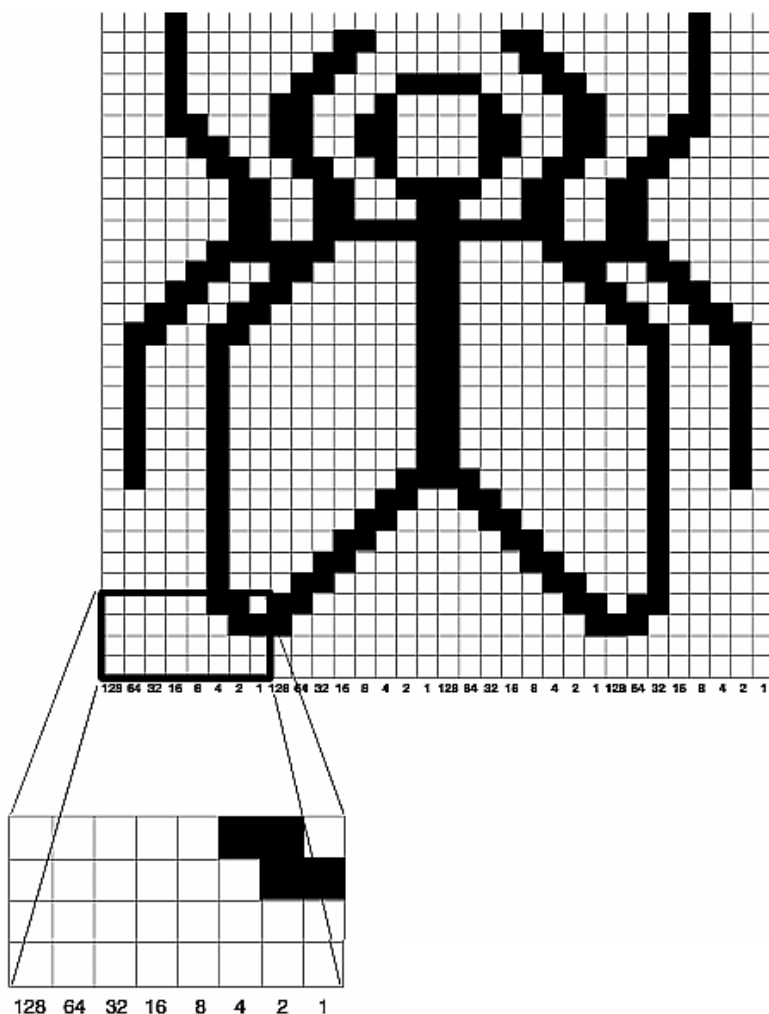


Figura 2.10 Construirea unui model pentru punctarea poligoanelor

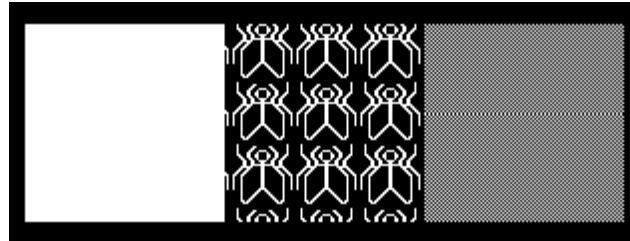


Figura 2.11 Poligoane punctate

Exemplul 2.6 Poligoane punctate

```
#include <windows.h>
#include <GL/gl.h>
#include <GL/glut.h>

void display(void)
{
    GLubyte fly[] = {
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
        0x03, 0x80, 0x01, 0xC0, 0x06, 0xC0, 0x03, 0x60,
        0x04, 0x60, 0x06, 0x20, 0x04, 0x30, 0x0C, 0x20,
        0x04, 0x18, 0x18, 0x20, 0x04, 0x0C, 0x30, 0x20,
        0x04, 0x06, 0x60, 0x20, 0x44, 0x03, 0xC0, 0x22,
        0x44, 0x01, 0x80, 0x22, 0x44, 0x01, 0x80, 0x22,
        0x44, 0x01, 0x80, 0x22, 0x44, 0x01, 0x80, 0x22,
        0x44, 0x01, 0x80, 0x22, 0x44, 0x01, 0x80, 0x22,
        0x66, 0x01, 0x80, 0x66, 0x33, 0x01, 0x80, 0xCC,
        0x19, 0x81, 0x81, 0x98, 0x0C, 0xC1, 0x83, 0x30,
        0x07, 0xe1, 0x87, 0xe0, 0x03, 0x3f, 0xfc, 0xc0,
        0x03, 0x31, 0x8c, 0xc0, 0x03, 0x33, 0xcc, 0xc0,
        0x06, 0x64, 0x26, 0x60, 0x0c, 0xcc, 0x33, 0x30,
        0x18, 0xcc, 0x33, 0x18, 0x10, 0xc4, 0x23, 0x08,
        0x10, 0x63, 0xC6, 0x08, 0x10, 0x30, 0x0c, 0x08,
        0x10, 0x18, 0x18, 0x08, 0x10, 0x00, 0x00, 0x08};

    GLubyte halftone[] = {
        0xAA, 0xAA, 0xAA, 0xAA, 0x55, 0x55, 0x55, 0x55,
        0xAA, 0xAA, 0xAA, 0xAA, 0x55, 0x55, 0x55, 0x55,
        0xAA, 0xAA, 0xAA, 0xAA, 0x55, 0x55, 0x55, 0x55,
```

```

0xAA, 0xAA, 0xAA, 0xAA, 0x55, 0x55, 0x55, 0x55,
0xAA, 0xAA, 0xAA, 0xAA, 0x55, 0x55, 0x55, 0x55,
0xAA, 0xAA, 0xAA, 0xAA, 0x55, 0x55, 0x55, 0x55,
0xAA, 0xAA, 0xAA, 0xAA, 0x55, 0x55, 0x55, 0x55,
0xAA, 0xAA, 0xAA, 0xAA, 0x55, 0x55, 0x55, 0x55,
0xAA, 0xAA, 0xAA, 0xAA, 0x55, 0x55, 0x55, 0x55,
0xAA, 0xAA, 0xAA, 0xAA, 0x55, 0x55, 0x55, 0x55,
0xAA, 0xAA, 0xAA, 0xAA, 0x55, 0x55, 0x55, 0x55,
0xAA, 0xAA, 0xAA, 0xAA, 0x55, 0x55, 0x55, 0x55,
0xAA, 0xAA, 0xAA, 0xAA, 0x55, 0x55, 0x55, 0x55,
0xAA, 0xAA, 0xAA, 0xAA, 0x55, 0x55, 0x55, 0x55,
0xAA, 0xAA, 0xAA, 0xAA, 0x55, 0x55, 0x55, 0x55,
0xAA, 0xAA, 0xAA, 0xAA, 0x55, 0x55, 0x55, 0x55,
0xAA, 0xAA, 0xAA, 0xAA, 0x55, 0x55, 0x55, 0x55,
0xAA, 0xAA, 0xAA, 0xAA, 0x55, 0x55, 0x55, 0x55};

```

```

glClear (GL_COLOR_BUFFER_BIT);
glColor3f (1.0, 1.0, 1.0);
/*desenează un pătrat solid fără modele, */
/* d */
glRectf (25.0, 25.0, 125.0, 125.0);
glEnable (GL_POLYGON_STIPPLE);
glPolygonStipple (fly);
glRectf (125.0, 25.0, 225.0, 125.0);
glPolygonStipple (halftone);
glRectf (225.0, 25.0, 325.0, 125.0);
glDisable (GL_POLYGON_STIPPLE);
glFlush ();

```

```

}

```

```

void init (void)

```

```

{

```

```

    glClearColor (0.0, 0.0, 0.0, 0.0);
    glShadeModel (GL_FLAT);

```

```

}

```

```

void reshape (int w, int h)

```

```

{

```

```

    glViewport (0, 0, (GLsizei) w, (GLsizei) h);
    glMatrixMode (GL_PROJECTION);
    glLoadIdentity ();
    gluOrtho2D (0.0, (GLdouble) w, 0.0, (GLdouble) h);

```

```
}

int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize (350, 150);
    glutCreateWindow (argv[0]);
    init ();
    glutDisplayFunc(display);
    glutReshapeFunc(reshape);
    glutMainLoop();
    return 0;
}
```

3. Transformări ale obiectelor 3D în OpenGL

3.1. Etapele transformării din spațiul 3D pe suprafața de afișare

OpenGL utilizează un sistem de coordonate 3D.

Pentru ca un obiect 3D să fie afișat pe ecran are loc o secvență de transformări aplicate punctelor prin care este definit acesta.(Figura 3.1)

- transformarea de modelare și vizualizare (Model View)
- transformarea de proiecție, însoțită de decupare la marginile volumului vizual canonic
- împărțirea perspectivă
- transformarea în poarta de vizualizare din fereastra curentă

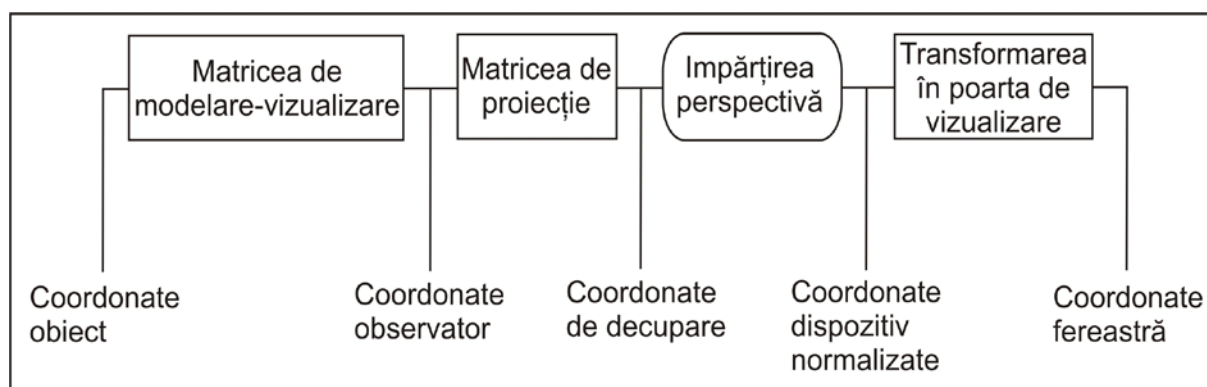


Figura 3.1 Secvența de transformări aplicate vârfurilor în OpenGL

Procesul de transformări necesar producerii imaginii dorite pentru a fi redată pe o suprafață de afișare este asemănător cu cel al efectuării unei fotografii.

Acești pași sunt următorii:

- Aranjarea scenei pentru a fi fotografiată în contextul dorit
– transformarea de modelare;
- Aranjarea aparatului de fotografiat și încadrarea scenei – transformare de vizualizare;
- Alegerea lentilelor aparatului sau modificarea zoom-ului – transformarea de proiecție;
- Determinarea dimensiunii imaginii finale
– transformarea în poarta de vizualizare(vizor).

3.2 Transformarea de modelare și vizualizare

Imaginea unui obiect se poate obține în două feluri:

- poziționând obiectul în fața unei camere de luat vederi fixe
- poziționând camera de luat vederi în fața obiectului fix

Prima operație corespunde unei transformări de modelare a obiectului. Cea de-a doua operație corespunde unei transformări de vizualizare. Deoarece ambele pot fi folosite pentru a obține o aceeași imagine, sunt tratate împreună, ca o singură transformare.

Transformarea de modelare are drept scop poziționarea obiectelor în scena 3D. Această transformare este necesară deoarece, în mod uzual, fiecare obiect este definit ca *obiect unitate* într-un *sistem de coordonate local*. De exemplu, un cub poate fi definit ca având latura de o unitate, centrat în originea unui sistem de coordonate carteziene 3D. Reprezentarea la mărimea dorită, poziționarea și orientare sa în scena 3D, care este definită într-un sistem de coordonate *global*, poate să presupună o transformare compusă din scalare și rotație față de originea sistemului de coordonate local, urmată de o translație. Prin această transformare se creează o *instanță* a cubului, de aceea transformarea de modelare se mai numește și *transformare de instanțiere*.

Transformare de modelare este o transformare compusă din transformări geometrice simple care poate fi definită ca produs matricial, folosind funcțiile de translație, rotație și scalare oferite de OpenGL.

Transformarea de vizualizare este determinată de poziția observatorului (camera de luat vederi), direcția în care privește acesta și direcția sus a planului de vizualizare. În mod implicit, observatorul este situat în originea sistemului de coordonate în care este descrisă scena 3D, direcția în care privește este direcția negativă a axei OZ, iar direcția sus a planului de vizualizare este direcția pozitivă a axei OY. Cu aceste valori implicite, transformarea de vizualizare este transformarea identică.

Funcția **gluLookAt()** permite modificarea parametrilor implicați ai transformării de vizualizare:

```
void gluLookAt(GLdouble eyex, GLdouble eyey, GLdouble eyez, GLdouble centerx,  
GLdouble centery, GLdouble centerz, GLdouble upx, GLdouble upy, GLdouble upz);
```

- **eyex, eyey, eyez** reprezintă poziția observatorului
- **centerx, centery, centerz** reprezintă direcția în care se privește
- **upx, upy, upz** reprezintă direcția vectorului „sus” al planului de vizualizare

Poziția observatorului reprezintă punctul de referință al vederii, R, iar direcția în care se privește este direcția normalei la planul de vizualizare, în R. Vectorul „sus” determină direcția pozitivă a axei verticale a sistemului de coordonate 2D atașat planului de vizualizare.

Sistemul de coordonate 2D atașat planului de vizualizare împreună cu normala la plan formează sistemul de coordonate 3D al planului de vizualizare, care în terminologia OpenGL este numit *sistemul de coordonate observator*.

Funcția **gluLookAt()** construiește *matricea transformării* din sistemul de coordonate *obiect* în sistemul de coordonate *observator* și o înmulțește la dreapta cu matricea curentă.

În OpenGL transformările de modelare și de vizualizare sunt exprimate printr-o singură matrice de transformare, care se obține înmulțind matricele celor două transformări. Ordinea de înmulțire a celor două matrici trebuie să corespundă ordinei în care ar trebui aplicate cele două transformări: mai întâi transformarea de modelare, apoi transformarea de vizualizare.

În OpenGL punctele 3D se reprezintă prin vectori coloană. Astfel, un punct (x, y, z) se reprezintă în coordonate omogene prin vectorul $[x_w \ y_w \ z_w \ w]^T$. Dacă A, B și C sunt 3 matrici de transformare care exprimă transformările de aplicat punctului în ordinea A, B, C, atunci secvența de transformări se exprimă matricial astfel:

$$[x_w y_w z_w w']^T = C * B * A * [x_w y_w z_w w]^T$$

Transformarea de modelare și vizualizare este o transformare compusă, reprezentată printr-o matrice VM, ce se obține înmulțind matricea transformării de vizualizare cu matricea transformării de modelare. Fie V și M aceste matrici. Atunci, $VM = V * M$.

Dacă coordonatele unui vârf în sistemul de coordonate obiect sunt reprezentate prin vectorul $[x_o \ y_o \ z_o \ w_o]^T$, atunci coordonatele vârfului în sistemul de coordonate observator („eye coordinates”) se obțin astfel:

$$[x_e \ y_e \ z_e \ w_e]^T = VM * [x_o \ y_o \ z_o \ w_o]^T$$

Matricea VM este aplicată automat și vectorilor normali.

3.3 Transformarea de proiecție

Matricea de proiecție este calculată în OpenGL în funcție de tipul de proiecție specificat de programator și parametrii care definesc volumul de vizualizare. Matricea de proiecție este matricea care transformă volumul vizual definit de programator într-un volum vizual canonic. Această transformare este aplicată vârfurilor care definesc primitivele geometrice în coordonate observator, rezultând *coordonate normalizate*. Primitivele reprezentate prin coordonate normalizate sunt apoi decupate la marginile volumului vizual canonic, de aceea coordonatele normalizate se mai numesc și *coordonate de decupare*. Volumul vizual canonic este un cub cu latura de 2 unități, centrat în originea sistemului coordonatelor de decupare. După aplicarea transformării de proiecție, orice punct 3D (din volumul vizual canonic) se proiectează în fereastra 2D printr-o proiecție ortografică ($x'=x$, $y'=y$) condiție necesară pentru aplicarea algoritmului z-buffer la producerea imaginii.

Dacă coordonatele unui vârf în sistemul de coordonate observator sunt reprezentate prin vectorul $[x_e \ y_e \ z_e \ w_e]^T$, iar P este matricea de proiecție, atunci coordonatele de decupare ale vârfului („clip coordinates”) se obțin astfel:

$$[x_e \ y_e \ z_e \ w_e]^T = P * [x_o \ y_o \ z_o \ w_o]^T$$

Prezentăm în continuare funcțiile OpenGL prin care pot fi definite proiecțiile și volumul de vizualizare:

```
void glFrustum(GLdouble left, GLdouble right, GLdouble bottom, GLdouble top, GLdouble near, GLdouble far);
```

Funcția definește o proiecție perspectivă cu centrul de proiecție în poziția observatorului (punctul de referință al vederii). Volumul de vizualizare (Figura 3.2) este delimitat prin „planul din față” și „planul din spate”, plane paralele cu planul de vizualizare, definite prin distanțele lor față de poziția observatorului (planul de vizualizare). Planul din față va fi folosit ca plan de proiecție. Lui i se atașează un sistem de coordonate 2D având axa verticală (sus) orientată ca și axa verticală a planului de vizualizare. Deschiderea camerei de luat vederi este determinată printr-o fereastră rectangulară, cu laturile paralele cu axele, definită în planul de proiecție.

- (left, bottom) și (right, top) reprezintă colțurile ferestrei din planul din față
- znear, zfar reprezintă distanțele de la poziția observatorului la planul din față, respectiv spate. Ambele distanțe trebuie să fie pozitive.

Dacă $left=right$ sau $bottom=top$ sau $znear=zfar$ sau $znear \leq 0$ sau $zfar \leq 0$ se semnalează eroare.

Colțurile ferestrei 2D din planul de proiecție, (left, bottom, -near) și (right, top, -near) sunt mapate pe colțurile stânga-jos și dreapta-sus ale ferestrei 2D din sistemul coordonatelor de decupare, adică (-1,-1,-1) și (1,1,-1).

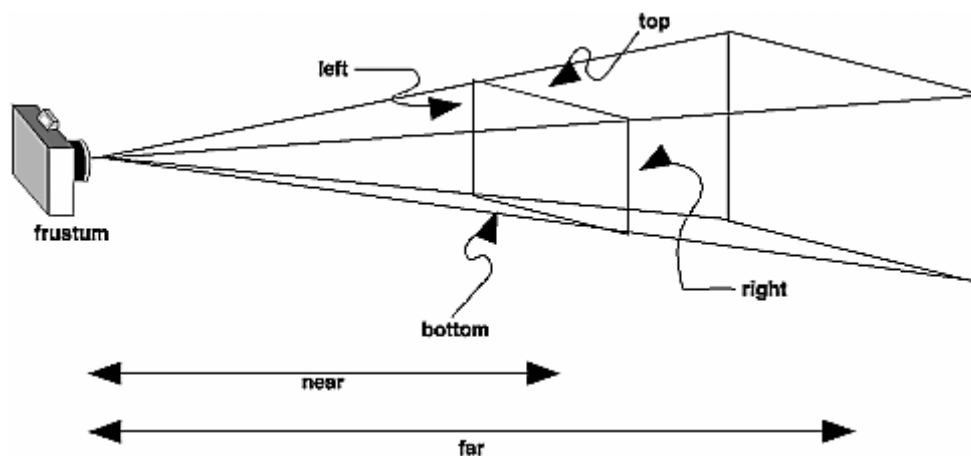


Figura 3.2 Volumul vizual – perspectivă

$$P = \begin{bmatrix} \frac{2 \cdot \text{near}}{\text{right} - \text{left}} & 0 & A & 0 \\ 0 & \frac{2 \cdot \text{near}}{\text{top} - \text{bottom}} & B & 0 \\ 0 & 0 & C & D \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

unde:

$$A = \frac{\text{right} + \text{left}}{\text{right} - \text{left}} \cdot B = \frac{\text{top} + \text{bottom}}{\text{top} - \text{bottom}}$$

$$C = -\frac{\text{far} + \text{near}}{\text{far} - \text{near}} \cdot D = \frac{2 \cdot \text{far} \cdot \text{near}}{\text{far} - \text{near}}$$

Funcția **glFrustum()** înmulțește matricea curentă cu matricea de proiecție rezultată și memorează rezultatul în matricea curentă. Astfel, dacă M este matricea curentă și P este matricea proiecției, atunci **glFrustum()** înlocuiește matricea M cu M*P.

O altă funcție care poate fi folosită pentru proiecția perspectivă este **gluPerspective()**:

`void gluPerspective(GLdouble fovy, GLdouble aspect, GLdouble near, GLdouble far);`

Funcția **gluPerspective()** creează un volum de vizualizare la fel ca și funcția **glFrustum()**, dar în acest caz el este specificat în alt mod. În cazul funcției **gluPerspective()** acesta se specifică prin unghiul de vizualizare în planul XOZ (deschiderea camerei de luat

vederi) și raportul dintre lățimea și înălțimea ferestrei definite în planul de aproape (pentru o fereastră pătrată acest raport este 1.0).

- **fovy** reprezintă unghiul de vizualizare în planul XOZ, care trebuie să fie în intervalul $[0.0, 180.0]$.
- **aspect** reprezintă raportul lățime / înălțime al laturilor ferestrei din planul de aproape; acest raport trebuie să corespundă raportului lățime / înălțime asociat porții de afișare. De exemplu, dacă **aspect** = 2.0 atunci unghiul de vizualizare este de două ori mai larg pe direcția x decât pe y. Dacă poarta de afișare este de două ori mai lată decât înălțimea atunci imaginea va fi afișată nedistorsionat.
- **near** și **far** reprezintă distanțele între observator și planele de decupare de-a lungul axei z negative. Întotdeauna trebuie să fie pozitivi.

Proiecția ortografică este specificată cu ajutorul funcției **glOrtho()**:

```
void glOrtho(GLdouble left, GLdouble right, GLdouble bottom, GLdouble top,
             GLdouble near, GLdouble far);
```

- **(left, bottom)** și **(right, top)** reprezintă colțurile ferestrei din planul din față
- **near, far** reprezintă distanțele de la poziția observatorului la planul din față, respectiv planul din spate.

Volumul de vizualizare este în acest caz un paralelipiped dreptunghic delimitat de planul din față și cel din spate, plane paralele cu planul de vizualizare (perpendiculare pe axa z), precum și de planele sus, jos, dreapta, stânga. Direcția de proiecție este dată de axa Z a sistemului de coordonate atașat planului de vizualizare.

Fereastra definită în planul din față, (left, bottom, -near) și (right, top, -near), este mapată pe fereastra 2D din sistemul coordonatelor de decupare (-1,-1,-1) și (1,1,-1).

Matricea de proiecție este în acest caz:

$$P = \begin{bmatrix} \frac{2}{right - left} & 0 & 0 & t_x \\ 0 & \frac{2}{top - bottom} & 0 & t_y \\ 0 & 0 & \frac{-2}{far - near} & t_z \\ 0 & 0 & 0 & -1 \end{bmatrix}$$

unde:

$$t_x = -\frac{right + left}{right - left}; t_y = -\frac{top + bottom}{top - bottom}; t_z = -\frac{far + near}{far - near}$$

Matricea curentă este înmulțită cu matricea de proiecție rezultată, rezultatul fiind depus în matrice curentă.

Tot pentru proiecția ortografică poate fi folosită și funcția **gluOrtho2D()** care definește matricea de proiecție ortografică în care near=-1 și far=1.

```
void gluOrtho2D(GLdouble left, GLdouble right, GLdouble bottom, GLdouble top);
```

unde: **(left, bottom)** și **(right, top)** reprezintă colțurile ferestrei din planul din față.

3.4 Împărțirea perspectivă

Prin această operație se obțin coordonatele 3D în spațiul coordonatelor de decupare, pornind de la coordonatele 3D omogene:

$$x_d = x_c/w_c, y_d = y_c/w_c, z_d = z_c/w_c$$

Coordonatele (x_d, y_d, z_d) sunt numite **coordonaate dispozitiv normalizate**.

Operația este denumită **împărțire perspectivă** deoarece numai în cazul unei proiecții perspectivă w_c este diferit de 1.

3.5 Transformarea în poarta de vizualizare

Această transformare se aplică coordonatelor dispozitiv normalizate pentru a se obține coordonate raportate la sistemul de coordonate al ferestrei curente de afișare. Este o transformare fereastră-poartă, în care fereastra este fereastra 2D din sistemul coordonatelor normalizate, având colțurile în $(-1, -1, -1); (1, 1, -1)$, iar poarta este un dreptunghi din fereastra de afișare care poate fi definit prin apelul funcției **glViewport()**.

```
void glViewport(GLint px, GLint py, GLsizei width, GLsizei height);
```

unde:

- **(px, py)** reprezintă coordonatele în fereastră ale colțului stânga jos al porții de afișare (în pixeli). Valorile implicite sunt (0.0).
- **width, height** reprezintă lățimea, respectiv înălțimea porții de afișare. Valorile implicite sunt date de lățimea și înălțimea ferestrei curente de afișare.

Fie (x_d, y_d, z_d) coordonate dispozitiv normalizate ale unui vârf și (x_w, y_w, z_w) coordonatele vârfului în fereastra de afișare.

Transformarea în poarta de vizualizare este definită astfel:

$$x_w = o_x + (width/2)x_d$$

$$y_w = o_y + (height/2)y_d$$

$$z_w = ((f-n)/2)z_d + (n+f)/2$$

unde:

- (ox, oy) reprezintă coordonatele centrului porții de afișare;
- **n, f** au valorile implicite 0.0, respectiv 1.0, dar pot fi modificate la valori cuprinse în intervalul $[0,1]$ folosind funcția **DepthRange()**. Dacă **n** și **f** au valorile implicite, atunci prin transformarea în poarta de vizualizare volumul vizual canonic se transformă în cubul cu colțurile de minim și maxim în punctele $(px, py, 0)-(px+width, py+height, 1)$
- calculul coordonatei z_w este necesară pentru comparațiile de adâncime în algoritmul z-buffer.

4. Iluminare în OpenGL

4.1 Introducere

Redarea luminilor în OpenGL este o sarcină relativ dificilă și asta deoarece sunt implicate setări în mai multe direcții:

- setări generale ale mediului OpenGL și ale interpretării luminii pe diferite fețe
- surse de lumini
- proprietățile surselor de lumini
- materiale
- normale la suprafață

Dacă una din aceste setări este greșită, atunci nu va merge nimic corect. De obicei, pentru a avea succes trebuie să programăm de la început foarte atent și foarte curat. De asemenea, este foarte puțin probabil să reușim să facem un efect interesant dacă nu înțelegem bine mecanismul luminilor.

4.2 Interpretarea modelului luminii naturale în OpenGL

Pentru început să ne imaginăm o scenă complexă din lumea reală. Ea este formată din obiecte cu diferite forme și texturi, dar și din multe lumini, reflexii de lumini sau umbre. Din punct de vedere fizic, lumina este o energie fonică care se mișcă prin spațiu aproape instantaneu. Când un obiect primește o rază de lumină, el păstrează o parte din energie pentru el, iar restul o împrăștie în mediu. Partea care o primește pentru el se transformă în căldură, iar partea care o emană va determina practic culoarea obiectului.

Cum sunt structurate toate acestea într-un model de lumini?

În OpenGL, se consideră că într-o scenă oarecare pot apărea mai multe **tipuri de lumini** de felul următor:

- lumina ambientală
- lumina difuză
- lumina speculară
- lumina emisă

4.2.1 Lumina ambientală

Aceasta este lumina care este prezentă în atmosferă “pur si simplu”. Nu putem spune despre ea nici că vine dintr-o anumită sursă, nici că este strălucitoare, nici că se reflectă de oglinda. Este provenită de la alte surse de lumini în urma reflexiilor repetate, nu se mai poate stabili exact unde a fost sursa și cum a ajuns lumina în locul respectiv.

Un exemplu bun de lumină ambientală este cea văzută într-un hol care primește lumina numai de la sursele din camerele vecine. Lumina poate fi amestecată din razele de lumina ce apar în urma reflexiilor. Lumina ambientală este singura care nu ne ajută să vedem formele obiectelor care au aceeași culoare. Motivul? Energia luminoasă este identică pentru fiecare față (nu depinde de orientare) și prin urmare culoarea văzută este exact aceeași.

4.2.2. Lumina Difuză

Aceasta este lumina provenită de la o sursă anume. Ea este formată din mai multe raze care se împrăștie în toate direcțiile. Acum lumina obiectului este calculată în funcție de direcția razei care luminează respectivul obiect. Sursa de lumină poate fi la infinit (imaginați-vă lumina solară într-o zi senină) sau poate fi apropiată de obiect (imaginați-vă un bec). Diferența dintre cele două tipuri de surse se observă și în direcția razelor. La soare razele vin paralele, la bec razele chiar se văd cum emană dintr-un punct fix. Lumina difuză se apropie cel mai mult de majoritatea luminilor din lumea reală, reprezintă clasa luminilor cu sursă și direcție. De reținut că poziția observatorului nu influențează culoarea obiectului în acest caz, cu alte cuvinte culoarea este transmisă la fel în toate direcțiile.

4.2.3. Lumina speculară

Aceasta este componenta care dă strălucire obiectelor. Se aseamănă cu lumina difuză, însa singura diferență este ca cea speculară este focalizată. Pentru a înțelege conceptul e bine să ne imaginăm o rază laser, sau un reflector de foarte bună calitate. Razele care pleacă din reflector trebuie să fie paralele. Obiectele resping această lumină tot într-un mod specular și prin urmare această lumină nu poate fi văzută din orice poziție. Ochiul trebuie să se afle chiar pe direcția ei.

4.2.4. Lumina emisă

Aceasta este lumina pe care un obiect o emite prin el însuși. In OpenGL aceasta nu mai devine și sursă pentru alte obiecte din jur. De regulă nu este nevoie să o folosim decât dacă vrem să creăm obiecte incandescente (lămpi, focuri, etc..)

4.3. Culorile luminii și materialelor în OpenGL

De remarcat că lumina are și culoare. Există în lumea reală și lumini verzi și lumini albastre, dar și lumini obișnuite albe. De cele mai multe ori, obiectul amestecă culorile primite de la lumina cu culorile sale naturale, putând rezulta chiar o culoare diferită de culoarea inițială a obiectului.

În OpenGL culoarea poate fi specificată prin cele 3 componente RGB. Acestea sunt niște valori reale între 0 și 1. De exemplu o lumină cu $(R,G,B)=(1,1,1)$ reprezintă o lumină albă, cea mai puternică. O lumină $(0,0,1)$ reprezintă o lumină albastruie. În OpenGL atât lumina cât și corpul au setate aceste valori pentru fiecare tip de lumină (ambientală, difuză și speculară). Lumina reflectată de un anumit obiect se obține înmulțind valoarea culorii cu valoarea luminii – bineînțeles ambele trebuie să se refere la același tip de lumină.

Din punct de vedere fizic, se pot întâmpla lucruri mult mai interesante cu raza de lumină în momentul în care aceasta atinge corpul. De ce nu mai e albă zăpada când se topește? Pentru că de regulă zăpada reflectă în totalitate lumina primită, dar dacă se topește începe să și absoarbă.

4.4. Pașii necesari pentru a seta un mediu de iluminare în OpenGL

Pentru ca să începem să folosim luminile avem nevoie de următoarele setări:

1. Inițializarea mediului

glEnable(GL_LIGHTING) : fără această funcție, OpenGL consideră că nu a auzit niciodată de conceptul de lumină.

2. Aprinderea luminilor

glEnable(GL_LIGHT0) : această funcție aprinde sursa de lumină cu numărul 1

glEnable(GL_LIGHT1) : această funcție aprinde sursa de lumină cu numărul 2

....

glEnable(GL_LIGHT8) : această funcție aprinde sursa de lumină cu numărul 8 (ultima)

Toate proprietățile luminii pe care le vom da în continuare se vor seta în una din aceste 8 lumini. Acesta este numărul maxim de lumini într-o scenă OpenGL. Cu toate acestea, mai există o sursă în scena introdusă automat chiar dacă nu sunt aprinse aceste lumini: **Lumina Ambientală Globală**

```
GLfloat lmodel_ambient[] = { 0.2, 0.2, 0.2, 1.0 };  
glLightModelfv(GL_LIGHT_MODEL_AMBIENT, lmodel_ambient);
```

Aceasta este lumina prin care se pot vedea obiectele chiar dacă noi nu adăugăm explicit lumina, ea definește o lumină ambientală de bază, mai palidă.

3. Setarea modelului de iluminare

Se pot seta diverși parametri ai iluminării:

glLightModeli(GL_LIGHT_MODEL_LOCAL_VIEWER, GL_TRUE) : această funcție setează poziția ochiului în (0,0,0). Avem nevoie de ea atunci când calculăm reflexiile speculare. De obicei, observatorul se consideră a fi la infinit astfel încât unghiul format de raza incidentă pe un punct și de linia observator-punct, depinde doar de raza incidentă.

glLightModeli(LIGHT_MODEL_TWO_SIDE, GL_TRUE): această funcție spune OpenGL-ului să realizeze toate calculele și pentru fețele considerate ca fiind cu spatele la lumină. De obicei aceste fețe sunt orientate spre interiorul corpurilor, deci nu avem nevoie de lumină pe ele. Însă dacă vrem să iluminăm și partea interioară a unei jumătăți de sferă, trebuie să setăm și acest parametru.

ShadeModel : Acesta nu face parte neapărat din modelul de iluminare, însă este destul de important în acest context. Se setează cu glShadeModel. Poate fi de două feluri: smooth sau flat. Modul flat pune aceiași culoare pe toată fața, în timp ce modul shade interpolează valorile din colțuri. Trebuie să fim atenți la cazul în care o lumină de tip spot cade pe centrul unui obiect fără să îi atingă colțurile. Dacă se întâmplă acest lucru, lumina va fi practic ignorată.

4.5. Setarea parametrilor obiectelor

După ce am parcurs pașii de mai sus, putem să începem să setăm parametrii efectivi ai luminilor. Ne alegem una din cele 8 surse de lumină, să zicem GL_LIGHT0, și vom opera asupra ei.

Un exemplu simplu - pentru a seta un parametru oarecare din cei ce urmează, se apelează un cod ca cel de mai jos:

```
GLfloat light_ambient[] = { 0.5, 0.5, 0.5, 1.0 };  
glLightfv(GL_LIGHT0, GL_AMBIENT, light_ambient);
```

Acest cod setează proprietatea GL_AMBIENT la valorile precizate pentru sursa de lumină GL_LIGHT0. Cei 4 parametri reprezintă valorile RGBA ale luminii – între 0 și 1. RGB este destul de clar ce înseamnă, alfa este folosit dacă vrem să facem, de exemplu, culoarea luminii mai importantă decât culoarea obiectului. Dacă obiectul are alfa mai mic decât 1.0, atunci culoarea naturală a obiectului va fi mai puțin importantă decât culoarea luminii atunci când se va calcula valoarea luminii reflectate.

Definirea completă a parametrilor luminii.

- **GL_AMBIENT, GL_DIFFUSE, GL_SPECULAR**

Aceste valori sunt toate definite ca în exemplul de mai sus și reprezintă culoarea luminii pentru cele 3 tipuri de lumini prezentate. Pentru a crea un efect realist e bine ca acestea să aibă aceleași valori – adică lumina difuză și ambientală să fie de aceeași culoare.

- **GL_POSITION**

Aceasta valoare e folosită pentru luminile care trebuie să aibă o sursă specificată: SPECULAR și DIFFUSE. Aceste tipuri de lumini au nevoie de o poziționare (trebuie să știm de unde pleacă lumina).

```
GLfloat light_position[] = { x,y,z,w};  
glLightfv(GL_LIGHT0, GL_POSITION, light_position);
```

Precum vedem, apelul este similar celui de la GL_AMBIENT. Dacă w, componenta a 4-a este 1 atunci se consideră că sursa este undeva la infinit pe direcția x, y, z (**Directional Light**). Altfel, ea se afla exact la poziția x, y, z (**Positional Light**).

- **GL_CONSTANT_ATTENUATION, GL_LINEAR_ATTENUATION, GL_QUADRATIC_ATTENUATION**

Acești parametri afectează factorii de atenuare a luminii. În lumea reală teoretic lumina se poate atenua. OpenGL permite atenuarea intensității luminii în funcție de distanța, pătratul distanței sau cubul distanței. Acești parametri se folosesc doar la luminile poziționale.

- **GL_SPOT_DIRECTION, GL_SPOT_EXPONENT, GL_SPOT_CUTOFF**

Acești parametri se folosesc la luminile poziționale, atât pentru componenta difuză cât și pentru componenta speculară. Dând acești 3 parametri, lumina emanată dintr-un punct (nu de la infinit) nu mai pleacă în toate direcțiile ci numai după o anumită orientare. Se dorește simularea luminilor de genul reflectoarelor de la concerte. GL_SPOT_DIRECTION reprezintă direcția (x, y, z) în care este orientat spotul, GL_SPOT_EXPONENT reprezintă concentrația luminii în interiorul spotului (poate fi constantă sau mai puternică spre centru și mai mică spre margini).

GL_SPOT_CUTOFF care reprezintă unghiul, sau deschiderea spotului. Inițial acesta este de 180 care înseamnă ($180 \times 2 = 360$) că spotul este de fapt tot o lumina emanată în toate direcțiile. Dar dacă, setăm alt unghi, suntem obligați să dăm o valoare între 0 și 90 de grade, adică să determinăm un spot realist (față de o anumită direcție – centru spotului).

4.6. Un exemplu de setare lumini

```
nr light_position[] = { 0.0, 100.0, 0.0, 1.0 }; //setează poziția  
nr light_ambient [] = { 0.9, 0.4, 0.0, 1.0 }; //setează componenta ambientală  
nr light_diffusion[] = { 0.9, 0.4, 0.0, 1.0 }; //setează componenta de difuzie  
nr light_specular[] = { 0.9, 0.4, 0.0, 1.0 }; //setează componenta speculară  
nr direction [] = { 1.0, -1.0, 1.0 }; //setează direcția spotului
```

```
glLightfv(GL_LIGHT1, GL_SPECULAR, light_specular);
glLightfv(GL_LIGHT1, GL_AMBIENT , light_ambient);
glLightfv(GL_LIGHT1, GL_DIFFUSE , light_difusion);
glLightfv(GL_LIGHT1, GL_POSITION, light_position);
glLightfv(GL_LIGHT1, GL_SPOT_DIRECTION, direction);
glLightf(GL_LIGHT1, GL_SPOT_CUTOFF, 45.0);
glLightf(GL_LIGHT1, GL_SPOT_EXPONENT, 3.0);
```

Acest exemplu setează o lumina roșiatică ($0.9 > 0.4$), poziționată la înălțimea 100 (observați că ultimul parametru al poziției nu este 0, deci avem lumina pozițională) și apoi îi definim și proprietățile de spot. Direcția este în jos, unghiul de deschidere totală va fi $45 \times 2 = 90$ și exponentul este 3 ca să concentrăm lumina către interiorul spotului. Matematic vorbind, intensitatea luminii într-o anumită parte a spotului, va fi $\cos(\text{unghi}(\text{raza în cadrul spotului, direcția generală a spotului}))^3$.

4.7. Setarea parametrilor materialului

Majoritatea caracteristicilor luminii trebuie să se regăsească în proprietățile de material pentru ca un obiect să reflecte o culoare. Aceasta va fi determinată pe baza proprietăților luminii dar și a proprietăților materialului.

Pentru luminile neambientale trebuie întotdeauna să se seteze normalele.

Caracteristicile materialelor sunt următoarele:

- GL_AMBIENT reprezintă partea ambientală a luminii ce va fi reflectată de obiect. De remarcat că este de forma (R, G, B, alfa), însă alfa nu contează întrucât în momentul când se va face blending, se ia în considerare o singură valoare alfa – cea a lui GL_DIFFUSE;
- GL_DIFFUSE reprezintă componenta difuză a culorii materialului. Este de forma (r, g, b, alfa);
- GL_AMBIENT_AND_DIFFUSE acest parametru este folosit deoarece în lumea reală sunt extrem de rare (practic nu există) obiectele care să aibă valori diferite al culorii reflexiei ambientale și difuze;
- GL_SPECULAR - aceasta este valoarea culorii speculare, oglindite de obiect. Acest parametru poate fi diferit, el reprezintă valoarea reflectată. De exemplu, la un diamant, culoarea reflectată nu e același lucru cu culoarea obiectului. O piatră prețioasă poate reflecta și raze colorate cu alte culori.
- GL_SHININESS este o valoare care ne spune cât de mult este focalizată raza speculară reflectată. Maximul este 128 și corespunde unei lumini concentrate practic la maximum într-o singură rază.

- `GL_EMISSION` este o setare prin care putem da incandescență unui obiect. Dând acest parametru, obiectul va apărea generând lumina prin el însuși, fără să o primească din alta parte.

4.8. Alte detalii de care trebuie să ținem cont

Matricea `ModelView` influențează poziția luminii doar în momentul când a fost setat prima oară `GL_POSITION`. Prin urmare, dacă modificăm poziția observatorului sau poziția unor obiecte și astfel modificăm `ModelView`, vom avea surpriza să nu găsim lumina la locul ei. Trebuie să resetăm valoarea `GL_POSITION` de fiecare dată când schimbăm `ModelView`. De asemenea, `ModelView` modifică și valorile normalelor. De aceea e bine să apelăm și `glEnable(GL_NORMALIZE)` pentru a fi siguri că normalele sunt puse corect (normalele se setează pentru fiecare vârf folosind de exemplu **`glNormal3d`**). Aceasta funcție poate fi apelată între `glBegin` și `glEnd`. Funcționează asemănător cu `glColor`: vârfurile definite după apel vor avea asociată ultima normală setată cu `glNormal`).

Dacă vrem să iluminăm doar anumite obiecte putem face asta setând `glEnable(GL_LIGHT0)` înainte de a desena obiectul în cauză.

OpenGL nu ne da nici un mecanism prin care să facem reflexii de lumini sau umbre. O rază reflectată nu mai devine sursă de lumină pentru alte obiecte.